

SDN Project Work:

Network Slicing with Ryu and Mininet

Network Slicing con Premium Links e Video Preemption

Abstract

Questo elaborato documenta la progettazione e l'implementazione di un sistema di Network Slicing in ambiente Software-Defined Networking. L'infrastruttura emulata con Mininet comprende sei switch OpenFlow e quattro host, interconnessi da link con parametri di banda e latenza differenziati. Due Premium Links a 6 Mbps e 3 ms di latenza affiancano i percorsi standard a 2 Mbps e 50 ms, creando corsie preferenziali per il traffico prioritario.

Il progetto si sviluppa in tre fasi. Il **Topology Slicing** partiziona la rete in due slice isolate tramite regole MAC-based, impedendo qualsiasi comunicazione tra host di slice diverse. Il **Service Slicing** introduce la differenziazione del traffico: i flussi video (UDP 9999) vengono instradati sui Premium Links mediante regole OpenFlow a priorità 200, mentre il traffico generico segue i percorsi standard a priorità 100. Il **Dynamic Slicing** aggiunge un meccanismo di Video Preemption che monitora il throughput in tempo reale con media mobile esponenziale e, alla rilevazione di traffico video, rimuove le allocazioni dinamiche (UDP 800, priorità 110) dai Premium Links per riservare il link al flusso prioritario.

Il monitoraggio in tempo reale è affidato a Grafana con backend InfluxDB: il controller scrive le metriche di throughput, stato dei link e preemption in un database a serie temporali, consultabile tramite dashboard grafiche. Il sistema è stato validato con test di connettività, misurazioni di banda con **iperf** e verifica delle regole OpenFlow sugli switch.

Indice

1	Introduzione	4
2	Architettura della Rete	5
2.1	Ambiente SDN e Strumenti Utilizzati	5
2.2	Topologia a Sei Switch	5
2.3	Ruolo degli Switch	6
2.4	I Premium Links	6
2.5	Configurazione dei Link	7
2.6	Port Mapping	7
2.7	Percorso dei Pacchetti nella Rete	8
2.8	Implementazione della Topologia (<code>topology.py</code>)	9
3	Topology Slicing	11
3.1	Principio di Funzionamento	11
3.2	Implementazione (<code>topology_slicing_controller.py</code>)	11
3.3	Verifica	14
4	Service Slicing con Premium Links	16
4.1	Obiettivo	16
4.2	Vantaggi dei Premium Links per il Video	16
4.3	Logica di Instradamento	16
4.4	Gestione ARP e Drop IPv6	17
4.5	Implementazione (<code>service_slicing_controller.py</code>)	17
4.6	Test di Verifica	19
5	Dynamic Slicing con Video Preemption	22
5.1	Concetto di Preemption	22
5.2	Stati del Sistema	22
5.3	Soglie di Intervento	23
5.4	Algoritmo di Preemption	23
5.5	Implementazione del Controller (<code>dynamic_slicing_controller.py</code>)	24
5.5.1	Inizializzazione e Stato dei Premium Links	24
5.5.2	Raccolta Statistiche e Calcolo del Throughput	25
5.5.3	Logica di Allocazione e Preemption	26
5.5.4	Installazione e Rimozione delle Regole Dinamiche	27
5.6	Esempio di Scenario	29
6	Test di Performance	30
6.1	Metodologia	30
6.2	Risultati Topology Slicing	30
6.3	Risultati Service Slicing	31
6.4	Stress Test	32
6.5	Risultati Dynamic Slicing	35
6.6	Verifica delle Flow Table	40
6.7	Analisi dei Pacchetti con Wireshark	41
6.8	Grafici dei Risultati	44

6.9	Generazione Avanzata di Traffico con D-ITG	45
7	Dashboard e Monitoraggio	49
7.1	Architettura del Monitoraggio	49
7.2	Scrittura delle Metriche nel Controller	49
7.3	Configurazione di InfluxDB	51
7.4	Configurazione di Grafana	52
7.5	Pannelli della Dashboard	53
7.6	REST API	53
7.7	Deployment con Docker Compose	53
7.8	Auto-Provisioning Grafana	55
8	Istruzioni Operative	57
8.1	Requisiti	57
8.2	Avvio del Sistema	57
8.3	Avvio del Monitoring	58
8.4	Test del Preemption	60
9	Considerazioni Finali	62
9.1	Risultati Ottenuti	62
9.2	Possibili Estensioni	62
	Appendice	63
	Struttura del Progetto	63

1 Introduzione

Nelle reti tradizionali tutto il traffico condivide le stesse risorse: un flusso video in tempo reale compete con un download in background per la stessa banda sugli stessi link, senza alcuna distinzione. Il Software-Defined Networking (SDN) permette di superare questo limite separando il piano di controllo da quello dati e rendendo la rete programmabile da un punto centralizzato.

Il **network slicing** usa questa separazione per dividere un'infrastruttura fisica in più "fette" logiche, ciascuna con le proprie regole di instradamento e la propria porzione di banda. Ogni slice funziona come una rete a sé, pur poggiando sullo stesso hardware.

In questo progetto il network slicing viene esteso con due meccanismi pensati per dare priorità al traffico video:

- **I Premium Links:** collegamenti diretti che aggirano i colli di bottiglia della rete standard, con banda tripla (6 vs 2 Mbps) e latenza ridotta (3 vs 50 ms per hop).
- **La Video Preemption:** un sistema che monitora il throughput dei Premium Links e, quando rileva un flusso video, rimuove il traffico generico per lasciare spazio al video.

Il progetto è organizzato in tre fasi:

1. **Topology Slicing** – La rete viene partizionata in due slice isolate. H1 comunica solo con H3, H2 solo con H4; qualsiasi traffico tra slice diverse viene scartato.
2. **Service Slicing con Premium Links** – Il traffico video (UDP 9999) viene instradato sui Premium Links tramite regole a priorità elevata, mentre il restante traffico segue i percorsi standard.
3. **Dynamic Slicing con Video Preemption** – Il controller monitora la rete continuamente e attiva la preemption quando rileva video, sottraendo i Premium Links al traffico generico per dedicarli al flusso prioritario.

L'intero sistema è stato implementato e testato in ambiente emulato con Mininet, utilizzando Ryu come controller SDN e Open vSwitch per gli switch OpenFlow. Le prestazioni sono state verificate con generatori di traffico (**iperf**) e analisi delle regole di flusso installate sugli switch.

2 Architettura della Rete

2.1 Ambiente SDN e Strumenti Utilizzati

Il progetto utilizza il paradigma SDN (Software-Defined Networking) con protocollo OpenFlow 1.3. In una rete SDN gli switch non prendono decisioni autonome di instradamento: si limitano a consultare le proprie *flow table* e, per ogni pacchetto in ingresso, applicano la prima regola corrispondente. Se nessuna regola corrisponde, il pacchetto viene inviato al controller tramite un messaggio **Packet-In**. Il controller può quindi analizzare il pacchetto e installare nuove regole sullo switch tramite messaggi **Flow-Mod**.

Ogni regola OpenFlow è composta da tre elementi: un **match** (i campi del pacchetto su cui operare), una **priorità** (per risolvere conflitti tra regole sovrapposte) e un insieme di **azioni** (output su una porta, drop, invio al controller).

Gli strumenti impiegati:

- **Mininet**: emulatore di reti SDN che crea host, switch e link virtuali in un unico ambiente Linux.
- **Ryu**: framework per controller SDN scritto in Python. Gestisce la comunicazione OpenFlow con gli switch e permette di installare regole via software.
- **Open vSwitch (OVS)**: switch virtuale compatibile con OpenFlow 1.3, utilizzato internamente da Mininet.
- **iperf**: generatore di traffico TCP/UDP per misurare banda e prestazioni.

2.2 Topologia a Sei Switch

La topologia comprende sei switch OpenFlow disposti in modo da creare percorsi ridondanti tra le sorgenti e gli utenti finali. La struttura segue uno schema a doppio percorso parallelo, con un livello di accesso (S1), un livello di distribuzione (S2/S3 per l'upper slice, S4/S5 per il lower) e un livello di consegna (S6).

La Figura 1 illustra la disposizione complessiva della rete.

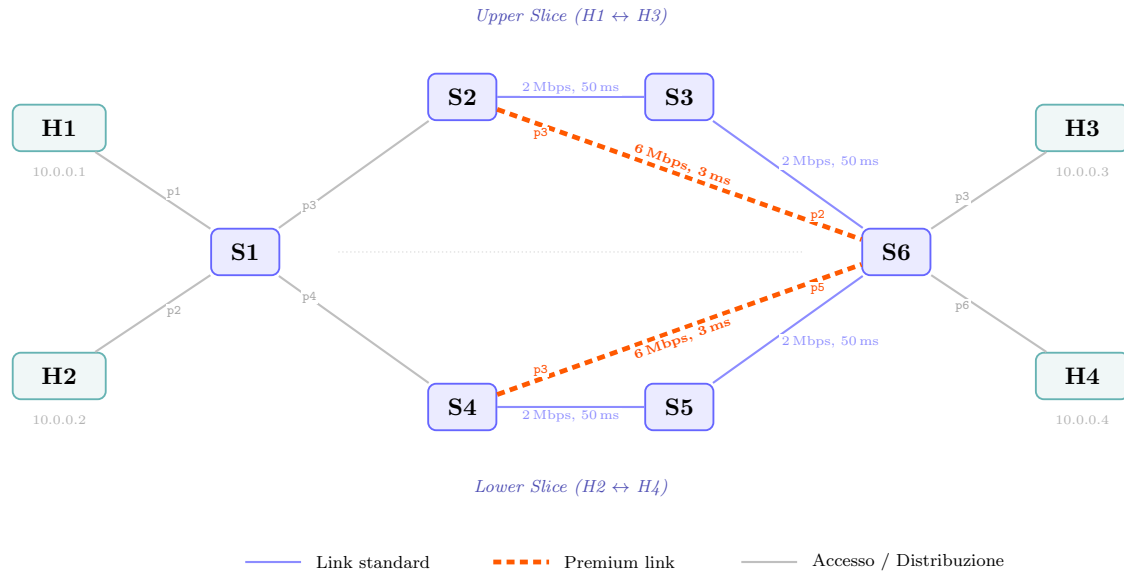


Figura 1: Topologia della rete a sei switch con Premium Links. Le etichette **pN** indicano le porte OpenFlow sugli switch S1 e S6. I link premium (tratteggiati) collegano S2 e S4 direttamente con S6, bypassando i colli di bottiglia S3 e S5.

S1 è il punto di accesso per le sorgenti H1 e H2, mentre S6 è l'hub di distribuzione verso H3 e H4. Gli switch S2/S3 costituiscono il percorso standard dell'upper slice, S4/S5 quello del lower slice. I Premium Links (S2–S6 e S4–S6) creano scorciatoie che bypassano i colli di bottiglia S3 e S5.

2.3 Ruolo degli Switch

La tabella seguente riassume il ruolo di ciascuno switch nella topologia.

Tabella 1: Ruolo funzionale degli switch nella topologia

Switch	Livello	Funzione
S1	Hub di Ingresso	Accesso sorgenti H1 (porta 1), H2 (porta 2), smistamento verso S2/S4
S2	Nodo Upper Slice	Percorso standard (porta 2, S3) o premium (porta 3, S6)
S3	Bottleneck Upper	Transito standard upper slice (2 Mbps, 50ms)
S4	Nodo Lower Slice	Percorso standard (porta 2, S5) o premium (porta 3, S6)
S5	Bottleneck Lower	Transito standard lower slice (2 Mbps, 50ms)
S6	Hub Distribuzione	Consegna a H3 (porta 3), H4 (porta 6), premium da S2/S4

2.4 I Premium Links

I Premium Links sono collegamenti diretti tra gli switch di distribuzione (S2, S4) e l'hub di consegna S6. Il percorso standard attraversa i link S2–S3–S6 (o S4–S5–S6), dove i segmenti S2–S3, S3–S6, S4–S5 e S5–S6 costituiscono colli di bottiglia a soli 2 Mbps con 50 ms di latenza ciascuno. I Premium Links aggirano questi segmenti collegando direttamente S2 e S4 con S6.

I vantaggi numerici:

- **Banda:** 6 Mbps contro 2 Mbps del percorso standard ($3\times$).
- **Latenza:** 3 ms contro i circa 100 ms accumulati lungo il percorso standard (50+50 ms per i due hop S2–S3 e S3–S6).
- **Isolamento:** il traffico premium non compete per la stessa capacità fisica dei link standard.

2.5 Configurazione dei Link

La tabella seguente riporta i parametri di banda e latenza di ciascun collegamento.

Tabella 2: Parametri di configurazione dei link

Tipo di Link	Collegamento	Banda (Mbps)	Latenza (ms)
Accesso Sorgente	S1–H1, S1–H2	15	1
Distribuzione	S1–S2, S1–S4	15	5
Standard (Upper)	S2–S3, S3–S6	2	50
Standard (Lower)	S4–S5, S5–S6	2	50
Premium	S2–S6, S4–S6	6	3
Consegna	S6–H3, S6–H4	8	5

I link standard (2 Mbps, 50 ms) sono volutamente i più lenti della topologia, così da rendere evidente il vantaggio dei Premium Links durante i test.

2.6 Port Mapping

La tabella seguente mostra la mappatura delle porte di ciascuno switch, necessaria per interpretare le regole OpenFlow installate dal controller.

Tabella 3: Mappatura delle porte per ciascuno switch

Switch	Porta 1	Porta 2	Porta 3
S1	H1 (accesso)	H2 (accesso)	S2 (distribuzione)
S1	Porta 4: S4 (distribuzione)		
S2	S1 (distribuzione)	S3 (standard)	S6 (PREMIUM)
S3	S2 (standard)	S6 (standard)	–
S4	S1 (distribuzione)	S5 (standard)	S6 (PREMIUM)
S5	S4 (standard)	S6 (standard)	–
S6	S3 (standard)	S2 (PREMIUM)	H3 (consegna)
S6	Porta 4: S5, Porta 5: S4 (PREMIUM), Porta 6: H4		

Ogni regola OpenFlow specifica una porta di output in base al MAC address di destinazione e, nel service slicing, anche in base al tipo di traffico.

2.7 Percorso dei Pacchetti nella Rete

Per comprendere il funzionamento delle regole OpenFlow, conviene seguire il percorso di un pacchetto specifico attraverso la rete. I due casi seguenti illustrano come lo stesso host sorgente (H1) raggiunge la stessa destinazione (H3) attraverso percorsi diversi, a seconda del tipo di traffico.

Caso A — Pacchetto video (UDP porta 9999). Un pacchetto UDP con porta destinazione 9999 inviato da H1 (10.0.0.1) a H3 (10.0.0.3) segue il percorso premium:

1. **S1** (porta 1 → porta 3): il pacchetto arriva dalla porta 1 (collegata a H1). La regola `priority=100, dl_dst=00:...:03, actions=output:3` lo inoltra verso S2. S1 non distingue il tipo di traffico: agisce solo in base al MAC di destinazione.
2. **S2** (porta 1 → porta 3): qui avviene la separazione. Due regole corrispondono al pacchetto: la regola di default (`priority=100, dl_dst=00:...:03, output:2`) e la regola video (`priority=200, udp, udp_dst=9999, output:3`). OpenFlow applica la regola con priorità più alta: il pacchetto esce dalla porta 3, il **Premium Link** verso S6.
3. **S6** (porta 2 → porta 3): il pacchetto arriva dalla porta 2 (Premium Link da S2). La regola `priority=100, dl_dst=00:...:03, actions=output:3` lo consegna a H3.

Hop totali: **3** (S1 → S2 → S6). Latenza attesa: $\sim 5 + 3 + 5 = 13$ ms.

Caso B — Pacchetto TCP generico (porta 5001). Un pacchetto TCP (porta 5001) dallo stesso H1 verso H3 segue il percorso standard:

1. **S1** (porta 1 → porta 3): identico al Caso A. S1 non distingue il tipo di traffico.
2. **S2** (porta 1 → porta 2): il pacchetto TCP non corrisponde alla regola video (non è UDP, non ha porta 9999). Viene applicata la regola di default (`priority=100, dl_dst=00:...:03, output:2`): il pacchetto esce dalla porta 2, il **percorso standard** verso S3.
3. **S3** (porta 1 → porta 2): S3 opera come semplice relay. La regola `priority=100, in_port=1, output:2` lo inoltra verso S6.
4. **S6** (porta 1 → porta 3): il pacchetto arriva dalla porta 1 (percorso standard da S3) e viene consegnato a H3 (porta 3).

Hop totali: **4** (S1 → S2 → S3 → S6). Latenza attesa: $\sim 5 + 50 + 50 + 5 = 110$ ms.

Caso C — Traffico dinamico (UDP porta 800, dynamic slicing). Con il dynamic slicing attivo, un pacchetto UDP porta 800 può seguire due percorsi diversi a seconda dello stato del sistema:

- **Premium Link libero** (allocazione attiva): la regola `priority=110, udp, udp_dst=800, output:3` su S2 cattura il pacchetto e lo dirige sul Premium Link. Poiché la priorità 110 è inferiore alla regola video (200), i flussi video passano sempre prima.

- **Dopo la preemption:** la regola a priorità 110 viene rimossa dal controller. Il pacchetto UDP 800 non corrisponde alla regola video (porta diversa) e viene catturato dalla regola di default (**priority=100, output:2**), che lo instrada sul percorso standard $S2 \rightarrow S3 \rightarrow S6$. Il cambio di percorso avviene senza alcun intervento sugli host.

La figura seguente riassume i percorsi del Caso A e del Caso B sulla topologia dell'upper slice ($H1 \rightarrow H3$):

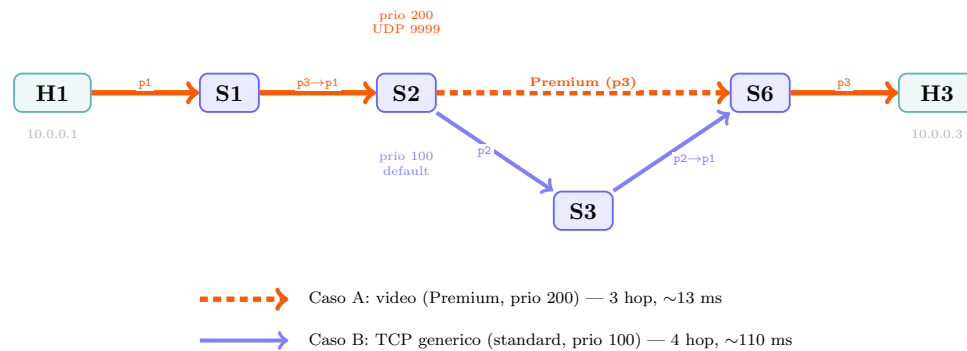


Figura 2: Percorso di un pacchetto video (arancione, Premium Link) e di un pacchetto TCP generico (blu, percorso standard) da H1 a H3. Su S2 la regola a priorità 200 devia il video sulla porta 3, mentre il traffico generico segue la porta 2 verso S3.

La differenza di percorso spiega i risultati misurati: il traffico video raggiunge una banda di ~5–6 Mbps con latenza di pochi millisecondi, mentre il traffico generico è limitato a ~1.8 Mbps con oltre 200 ms di RTT.

2.8 Implementazione della Topologia (topology.py)

La topologia viene definita come classe Mininet con porte esplicitamente assegnate. I MAC address sono statici per coerenza con le regole dei controller.

```

1 # Indirizzi MAC statici
2 H1_MAC = '00:00:00:00:00:01'
3 H2_MAC = '00:00:00:00:00:02'
4 H3_MAC = '00:00:00:00:00:03'
5 H4_MAC = '00:00:00:00:00:04'
6
7 # Parametri di banda (Mbps) e latenza
8 ACCESS_BW = 15; STANDARD_BW = 2; PREMIUM_BW = 6; DELIVERY_BW = 8
9 STANDARD_DELAY = '50ms'; PREMIUM_DELAY = '3ms'
10
11 class PremiumLinkTopology(Topo):
12     def build(self):
13         # Host con MAC statici
14         h1 = self.addHost('H1', ip='10.0.0.1/24', mac=H1_MAC)
15         h2 = self.addHost('H2', ip='10.0.0.2/24', mac=H2_MAC)
16         h3 = self.addHost('H3', ip='10.0.0.3/24', mac=H3_MAC)
17         h4 = self.addHost('H4', ip='10.0.0.4/24', mac=H4_MAC)
18
19         # Switch OpenFlow 1.3
20         s1 = self.addSwitch('s1', dpid='1', protocols='OpenFlow13')
21         s2 = self.addSwitch('s2', dpid='2', protocols='OpenFlow13')

```

```
22 # ... (s3, s4, s5, s6 analoghi)
```

Listing 1: Costanti e creazione host/switch in topology.py

```
1 # Upper Slice: H1 -> S1 -> S2 -> S3 -> S6 -> H3
2 self.addLink(h1, s1, bw=ACCESS_BW, delay='1ms', port1=1, port2
=1)
3 self.addLink(s1, s2, bw=ACCESS_BW, delay='5ms', port1=3, port2
=1)
4 self.addLink(s2, s3, bw=STANDARD_BW, delay=STANDARD_DELAY,
5 port1=2, port2=1) # Collo di bottiglia
6 self.addLink(s3, s6, bw=STANDARD_BW, delay=STANDARD_DELAY,
7 port1=2, port2=1)
8 self.addLink(s6, h3, bw=DELIVERY_BW, delay='5ms', port1=3, port2
=1)
9
10 # Lower Slice: H2 -> S1 -> S4 -> S5 -> S6 -> H4
11 self.addLink(h2, s1, bw=ACCESS_BW, delay='1ms', port1=1, port2
=2)
12 self.addLink(s1, s4, bw=ACCESS_BW, delay='5ms', port1=4, port2
=1)
13 self.addLink(s4, s5, bw=STANDARD_BW, delay=STANDARD_DELAY,
14 port1=2, port2=1)
15 self.addLink(s5, s6, bw=STANDARD_BW, delay=STANDARD_DELAY,
16 port1=2, port2=4)
17 self.addLink(s6, h4, bw=DELIVERY_BW, delay='5ms', port1=6, port2
=1)
18
19 # Premium Links: bypass dei colli di bottiglia
20 self.addLink(s2, s6, bw=PREMIUM_BW, delay=PREMIUM_DELAY,
21 port1=3, port2=2) # S2:3 <-> S6:2
22 self.addLink(s4, s6, bw=PREMIUM_BW, delay=PREMIUM_DELAY,
23 port1=3, port2=5) # S4:3 <-> S6:5
```

Listing 2: Creazione dei link con porte esplicite

```
1 def run_topology():
2     topo = PremiumLinkTopology()
3     net = Mininet(
4         topo=topo,
5         switch=OVSKernelSwitch,
6         controller=RemoteController('c0', ip='127.0.0.1', port=6653),
7         link=TCLink,
8         autoSetMacs=False # MAC statici definiti negli addHost
9     )
10    net.start()
11    CLI(net)
12    net.stop()
```

Listing 3: Avvio della rete con controller remoto

Il parametro `autoSetMacs=False` è necessario perché i MAC address sono assegnati esplicitamente negli `addHost`. Le porte sono fissate tramite `port1/port2` per mantenere la corrispondenza con le regole dei controller.

3 Topology Slicing

3.1 Principio di Funzionamento

Il topology slicing consiste nel partizionare la rete in sottoinsiemi completamente isolati: gli host di una slice non possono comunicare con quelli di un'altra, come se si trovassero su reti fisiche distinte.

La rete viene divisa in due slice:

- **Upper Slice:** $H1 \leftrightarrow H3$, percorso $S1 \rightarrow S2 \rightarrow S3 \rightarrow S6$ (standard) oppure $S1 \rightarrow S2 \rightarrow S6$ (premium).
- **Lower Slice:** $H2 \leftrightarrow H4$, percorso $S1 \rightarrow S4 \rightarrow S5 \rightarrow S6$ (standard) oppure $S1 \rightarrow S4 \rightarrow S6$ (premium).

I due sottoinsiemi non condividono alcun percorso oltre lo switch S1 (che smista il traffico verso S2 o S4 in base alla slice di appartenenza) e lo switch S6 (che consegna i pacchetti all'host di destinazione). Le regole OpenFlow fanno sì che ogni pacchetto venga inoltrato solo verso porte della propria slice; un pacchetto cross-slice viene scartato.

3.2 Implementazione (topology_slicing_controller.py)

Il controller utilizza un approccio **reattivo**: le regole vengono installate al primo pacchetto di ogni flusso. Le richieste ARP broadcast vengono confinate alla slice di appartenenza per impedire agli host di slice diverse di scoprirsi.

Le slice sono definite come insiemi di MAC address:

```

1 class TopologySlicingController(app_manager.RyuApp):
2     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
3
4     # Definizione delle slice tramite insiemi di MAC address
5     UPPER_SLICE = {'00:00:00:00:00:01', '00:00:00:00:00:03'} # H1, H3
6     LOWER_SLICE = {'00:00:00:00:00:02', '00:00:00:00:00:04'} # H2, H4
7
8     # Tabella di routing statico: {dpid: {mac_dst: porta_uscita}}
9     # Solo percorsi standard, i Premium Links NON compaiono
10    MAC_TO_PORT = {
11        1: {'00:00:00:00:00:01': 1, '00:00:00:00:00:02': 2, # H1, H2
12        locali
13            '00:00:00:00:00:03': 3, '00:00:00:00:00:04': 4}, # H3->S2,
14        H4->S4
15        2: {'00:00:00:00:00:01': 1, '00:00:00:00:00:03': 2}, # verso S1
16        , verso S3
17        3: {'00:00:00:00:00:01': 1, '00:00:00:00:00:03': 2}, # verso S2
18        , verso S6
19        4: {'00:00:00:00:00:02': 1, '00:00:00:00:00:04': 2}, # verso S1
20        , verso S5
21        5: {'00:00:00:00:00:02': 1, '00:00:00:00:00:04': 2}, # verso S4
22        , verso S6
23        6: {'00:00:00:00:00:01': 1, '00:00:00:00:00:02': 4, # S3, S5 (
24        standard)
25            '00:00:00:00:00:03': 3, '00:00:00:00:00:04': 6}, # H3, H4
26        locali
27    }

```

Listing 4: Definizione delle slice e tabella di routing

I metodi `_get_slice` e `_is_same_slice` implementano la logica di isolamento:

```

1  def _get_slice(self, mac):
2      """Restituisce 'upper', 'lower' o None per MAC sconosciuti."""
3      if mac in self.UPPER_SLICE:
4          return 'upper'
5      elif mac in self.LOWER_SLICE:
6          return 'lower'
7      return None    # MAC non appartenente a nessuna slice
8
9  def _is_same_slice(self, mac1, mac2):
10     """True solo se entrambi i MAC appartengono alla stessa slice."""
11
12     slice1 = self._get_slice(mac1)
13     slice2 = self._get_slice(mac2)
14     if slice1 is None or slice2 is None:
15         return False    # MAC sconosciuto -> blocca (fail-closed)
16     return slice1 == slice2    # True solo se upper==upper o lower==
lower

```

Listing 5: Verifica appartenenza alla stessa slice

L'handler `packet_in` implementa la logica principale. Per ogni pacchetto non matchato da regole esistenti, verifica la slice e decide se installare una regola o scartare il pacchetto:

```

1  @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
2  def packet_in_handler(self, ev):
3      """Gestisce ogni pacchetto non matchato da regole esistenti."""
4      msg = ev.msg
5      datapath = msg.datapath    # lo switch che ha generato il
packet-in
6      parser = datapath.ofproto_parser
7      dpid = datapath.id    # ID dello switch (1-6)
8      in_port = msg.match['in_port']    # porta da cui e' arrivato il
pacchetto
9
10     # Parsing dell'header Ethernet
11     pkt = packet.Packet(msg.data)
12     eth = pkt.get_protocol(ethernet.ethernet)
13     if eth is None:
14         return
15     src_mac = eth.src    # MAC sorgente
16     dst_mac = eth.dst    # MAC destinazione
17
18     # Ignora pacchetti LLDP (usati da OpenFlow per discovery)
19     if eth.ethertype == 0x88CC:
20         return
21
22     # Broadcast (ARP): confina alla slice della sorgente
23     if dst_mac == 'ff:ff:ff:ff:ff:ff':
24         self._handle_broadcast(datapath, in_port, src_mac, msg)
25         return
26
27     # CROSS-SLICE CHECK: blocca comunicazione tra slice diverse
28     if not self._is_same_slice(src_mac, dst_mac):

```

```

29         self.logger.warning("S%d: BLOCCATO %s -> %s", dpid, src_mac,
dst_mac)
30         return # Drop silenzioso: il pacchetto viene scartato
31
32     # Forwarding intra-slice: cerca la porta di uscita e installa
regola
33     if dpid in self.MAC_TO_PORT and dst_mac in self.MAC_TO_PORT[dpid
]:
34         out_port = self.MAC_TO_PORT[dpid][dst_mac]
35         match = parser.OFPMatch(eth_src=src_mac, eth_dst=dst_mac)
36         actions = [parser.OFPActionOutput(out_port)]
37         # Installazione REATTIVA: la regola viene creata al primo
pacchetto
38         self._add_flow(datapath, 10, match, actions, idle_timeout
=300)
39         self._send_packet(datapath, msg, actions) # inoltra il
pacchetto

```

Listing 6: Handler packet-in del topology slicing

Il broadcast ARP viene propagato solo sulle porte della slice della sorgente. La struttura SLICE_PORTS definisce, per ogni switch, quali porte appartengono a ciascuna slice:

```

1     # Per ogni switch: porte che appartengono a ciascuna slice
2     # Le porte premium (S2:3, S4:3, S6:2, S6:5) NON compaiono
3     SLICE_PORTS = {
4         1: {'upper': [1, 3], 'lower': [2, 4]}, # H1+S2, H2+S4
5         2: {'upper': [1, 2], 'lower': []},      # solo upper (no
premium)
6         3: {'upper': [1, 2], 'lower': []},
7         4: {'upper': [], 'lower': [1, 2]},      # solo lower (no
premium)
8         5: {'upper': [], 'lower': [1, 2]},
9         6: {'upper': [1, 3], 'lower': [4, 6]}, # S3+H3, S5+H4
10    }
11
12    def _handle_broadcast(self, datapath, in_port, src_mac, msg):
13        """Propaga ARP solo sulle porte della slice della sorgente."""
14        dpid = datapath.id
15        parser = datapath.ofproto_parser
16        src_slice = self._get_slice(src_mac) # 'upper' o 'lower'
17        if src_slice is None:
18            return # MAC sconosciuto: non propagare
19
20        # Porte della slice, escludendo la porta di arrivo
21        slice_ports = self.SLICE_PORTS[dpid].get(src_slice, [])
22        out_ports = [p for p in slice_ports if p != in_port]
23        if not out_ports:
24            return
25
26        # Invio su tutte le porte della slice (esclusa quella di
ingresso)
27        actions = [parser.OFPActionOutput(port) for port in out_ports]
28        self._send_packet(datapath, msg, actions)

```

Listing 7: ARP Confinement: broadcast limitato alla slice

All'avvio, lo `switch_features_handler` installa su ogni switch la regola `table-miss` (priorità 0, invia al controller) e la regola di drop IPv6 (priorità 5, scarta pacchetti con EtherType 0x86DD).

3.3 Verifica

Utilizzando il comando `pingall` di Mininet, ci si aspetta che solo gli host della stessa slice riescano a comunicare tra loro.

```
1 mininet> pingall
2 *** Ping: testing ping reachability
3 H1 -> X H3 X          # H1 raggiunge solo H3 (upper slice)
4 H2 -> X X H4          # H2 raggiunge solo H4 (lower slice)
5 H3 -> H1 X X          # H3 raggiunge solo H1 (upper slice)
6 H4 -> X H2 X          # H4 raggiunge solo H2 (lower slice)
7 *** Results: 66% dropped (4/12 received)
```

Listing 8: Test del topology slicing

Il risultato atteso è **4 ping riusciti su 12 possibili**: ogni host riesce a raggiungere solo l'host della propria slice ($H1 \leftrightarrow H3$, $H2 \leftrightarrow H4$). Gli 8 ping falliti corrispondono ai tentativi di comunicazione tra slice diverse, correttamente bloccati dall'assenza di regole di forwarding.

```
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
H1 -> X H3 X
H2 -> X X H4
H3 -> H1 X X
H4 -> X H2 X
*** Results: 66% dropped (4/12 received)
mininet> █
```

Figura 3: Risultato del comando `pingall` con topology slicing: solo le coppie intra-slice ($H1 \leftrightarrow H3$, $H2 \leftrightarrow H4$) comunicano.

```

mininet> H1 ping -c 4 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=296 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=240 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=245 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=227 ms

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3018ms
rtt min/avg/max/mdev = 226.854/252.015/296.053/26.302 ms
mininet> H1 ping -c 4 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable

--- 10.0.0.4 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3093ms
pipe 4
mininet>

```

Figura 4: Test di connettività intra-slice e cross-slice. Il ping H1→H3 riesce (stessa slice), mentre H1→H4 fallisce (slice diverse).

```

vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ sudo ovs-ofctl dump-flows s6 --protocols=
OpenFlow13
cookie=0x0, duration=8.830s, table=0, n_packets=2, n_bytes=140, idle_timeout=300, priority=10,dl_src=00:00:
00:00:00:03,dl_dst=00:00:00:00:00:00:01 actions=output:"s6-eth1"
cookie=0x0, duration=8.574s, table=0, n_packets=1, n_bytes=42, idle_timeout=300, priority=10,dl_src=00:00:0
0:00:00:01,dl_dst=00:00:00:00:00:00:03 actions=output:"s6-eth3"
cookie=0x0, duration=20.422s, table=0, n_packets=88, n_bytes=13028, priority=5,ipv6 actions=drop
cookie=0x0, duration=20.422s, table=0, n_packets=14, n_bytes=644, priority=0 actions=CONTROLLER:65535

```

Figura 5: Flow table dello switch S6 con topology slicing: le regole a priorità 10 instradano il traffico tra H1 e H3 (upper slice) in base ai MAC di sorgente e destinazione, con `idle_timeout=300s`.

4 Service Slicing con Premium Links

4.1 Obiettivo

A differenza del topology slicing, che si occupa solo di isolare le slice, il service slicing differenzia il traffico *all'interno* della stessa slice in base al tipo di servizio. L'obiettivo è instradare il traffico video sui Premium Links e il traffico generico (ping, TCP, download) sui percorsi standard.

Con il service slicing tutti gli host possono comunicare tra loro (H1 può raggiungere H3, H2 può raggiungere H4, e viceversa), ma il percorso seguito dipende dal tipo di traffico, non dall'host di origine.

4.2 Vantaggi dei Premium Links per il Video

I due percorsi hanno prestazioni molto diverse per lo streaming video.

Tabella 4: Confronto tra percorso standard e premium per il traffico video

Metrica	Standard	Premium
Banda disponibile	~2 Mbps	~6 Mbps
Latenza end-to-end	~100 ms (50+50)	~3 ms
Numero di hop	4 (S1-S2-S3-S6)	3 (S1-S2-S6 o S1-S4-S6)
Congestione	Alta (link condiviso)	Bassa (link dedicato)
Qualità video	SD (480p)	HD (720p/1080p)
Buffering	Frequente	Raro

Con 6 Mbps di banda dedicata, un flusso video in HD a 720p (che richiede tipicamente 3–5 Mbps) può essere trasmesso senza problemi. Sul percorso standard, con soli 2 Mbps, lo stesso flusso subirebbe continue interruzioni e dovrebbe essere degradato a qualità SD.

4.3 Logica di Instradamento

L'instradamento nel service slicing funziona con due livelli di priorità nelle regole OpenFlow:

- **Priorità 200:** regole per il traffico video. Queste regole matchano pacchetti UDP sulla porta 9999 (la porta convenzionalmente usata per il traffico video nel nostro sistema) e li dirigono verso i premium links.
- **Priorità 100:** regole di default per tutto il restante traffico. Queste regole matchano solo sull'indirizzo MAC di destinazione e instradano i pacchetti sul percorso standard.

Il meccanismo di priorità di OpenFlow garantisce che, quando un pacchetto corrisponde a più regole, venga applicata quella con priorità maggiore. Quindi un pacchetto UDP porta 9999 verrà sempre instradato sul premium link, indipendentemente dalla regola di default.

4.4 Gestione ARP e Drop IPv6

Oltre alle regole di instradamento, i controller service slicing e dynamic slicing installano regole aggiuntive comuni a tutti gli switch:

- **ARP flooding controllato (priorità 50):** i pacchetti ARP broadcast non possono usare OFPP_FLOOD perché i premium links creano cicli (S2-S6 e S4-S6) che causerebbero broadcast storm. Il controller installa regole ARP per-porta su ogni switch, specificando esattamente quali porte di uscita usare per ogni porta di ingresso, *escludendo* sempre le porte premium (S2:3, S4:3, S6:2, S6:5). In questo modo la risoluzione ARP avviene senza loop.
- **Drop IPv6 (priorità 5):** i pacchetti IPv6 (EtherType 0x86DD) vengono scartati silenziosamente su tutti gli switch. La rete usa esclusivamente IPv4 e i pacchetti IPv6 generati automaticamente dal kernel (Neighbor Discovery, Router Solicitation) causerebbero carico inutile sul controller.
- **Table-miss (priorità 0):** regola di fallback che invia al controller qualsiasi pacchetto non corrispondente ad altre regole.

4.5 Implementazione (service_slicing_controller.py)

Le regole vengono installate in modo **proattivo**: al momento della connessione di ogni switch, il controller configura immediatamente tutte le regole necessarie, evitando la latenza aggiuntiva del primo pacchetto.

La mappa delle porte usa nomi descrittivi per migliorare la leggibilità:

```

1 class ServiceSlicingController(app_manager.RyuApp):
2     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
3
4     def __init__(self, *args, **kwargs):
5         super().__init__(*args, **kwargs)
6         self.MAC_H1 = "00:00:00:00:00:01"
7         self.MAC_H3 = "00:00:00:00:00:03"
8         # ... MAC_H2, MAC_H4 analoghi
9
10        self.PORTS = {
11            2: {'to_s1': 1, 'to_s3_standard': 2, 'to_s6_premium': 3},
12            4: {'to_s1': 1, 'to_s5_standard': 2, 'to_s6_premium': 3},
13            6: {'from_s3_standard': 1, 'from_s2_premium': 2, 'H3': 3,
14               'from_s5_standard': 4, 'from_s4_premium': 5, 'H4': 6},
15            # ... S1, S3, S5 analoghi
16        }
17        self.VIDEO_PRIORITY = 200
18        self.DEFAULT_PRIORITY = 100
19        self.ARP_PRIORITY = 50

```

Listing 9: Mappatura porte e priorità nel service slicing controller

Lo switch S2 è il punto dove avviene la separazione del traffico: i pacchetti video (UDP 9999) vengono diretti sulla porta 3 (premium), il resto sulla porta 2 (standard). La priorità 200 della regola video vince sulla regola default a 100.

```

1 def install_s2_rules(self, datapath):
2     parser = datapath.ofproto_parser

```

```

3      ports = self.PORTS[2]
4
5      # Regola VIDEO: UDP 9999 -> link premium (priorita 200)
6      match_video_h3 = parser.OFPMatch(
7          eth_type=0x0800, ip_proto=17, udp_dst=9999,
8          eth_dst=self.MAC_H3
9      )
10     actions = [parser.OFPACTIONOutput(ports["to_s6_premium"])]
11     self.add_flow(datapath, self.VIDEO_PRIORITY, match_video_h3,
12         actions)
13
14     # Regola DEFAULT: tutto il resto -> link standard (priorita 100)
15     match_default_h3 = parser.OFPMatch(eth_dst=self.MAC_H3)
16     actions = [parser.OFPACTIONOutput(ports["to_s3_standard"])]
17     self.add_flow(datapath, self.DEFAULT_PRIORITY, match_default_h3,
18         actions)
19
20     # Ritorno: da S3 e da S6 premium -> verso S1
21     match = parser.OFPMatch(in_port=ports["to_s3_standard"])
22     self.add_flow(datapath, self.DEFAULT_PRIORITY, match,
23         [parser.OFPACTIONOutput(ports["to_s1"])]])
24     match = parser.OFPMatch(in_port=ports["to_s6_premium"])
25     self.add_flow(datapath, self.DEFAULT_PRIORITY, match,
26         [parser.OFPACTIONOutput(ports["to_s1"])]])

```

Listing 10: Regole di S2: separazione video/standard

Le regole ARP non possono usare OFPP_FLOOD perché i Premium Links creano cicli. Il controller installa regole per-porta che escludono le porte premium:

```

1      def install_arp_flood(self, datapath):
2          parser = datapath.ofproto_parser
3          dpid = datapath.id
4
5          # Porte premium escluse: S2:3, S4:3, S6:2, S6:5
6          arp_forward = {
7              1: {1: [3, 4], 2: [3, 4], 3: [1, 2, 4], 4: [1, 2, 3]},
8              2: {1: [2], 2: [1]}, # porta 3 (premium) esclusa
9              3: {1: [2], 2: [1]},
10             4: {1: [2], 2: [1]}, # porta 3 (premium) esclusa
11             5: {1: [2], 2: [1]},
12             6: {1: [3, 4, 6], 3: [1, 4, 6], 4: [1, 3, 6], 6: [1, 3, 4]},
13         }
14
15         if dpid in arp_forward:
16             for in_port, out_ports in arp_forward[dpid].items():
17                 match = parser.OFPMatch(
18                     eth_type=0x0806, in_port=in_port) # ARP
19                 actions = [parser.OFPACTIONOutput(p) for p in out_ports]
20                 self.add_flow(datapath, self.ARP_PRIORITY, match,
21                     actions)

```

Listing 11: ARP flooding controllato (senza porte premium)

Lo `switch_features_handler` installa le regole comuni (table-miss, ARP, drop IPv6) e poi le regole specifiche in base al DPID dello switch:

```

1      @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2      def switch_features_handler(self, ev):
3          """Chiamato quando uno switch completa l'handshake OpenFlow."""

```

```

4      datapath = ev.msg.datapath
5      dpid = datapath.id # ID dello switch appena connesso
6
7      # Regole comuni a TUTTI gli switch
8      self.install_table_miss(datapath) # prio 0: fallback al
controller
9      self.install_arp_flood(datapath) # prio 50: ARP senza premium
10     self.install_ipv6_drop(datapath) # prio 5: scarta IPv6
11
12     # Regole specifiche per ciascuno switch (video/standard/ritorno)
13     if dpid == 1: self.install_s1_rules(datapath)
14     elif dpid == 2: self.install_s2_rules(datapath)
15     elif dpid == 3: self.install_s3_rules(datapath)
16     elif dpid == 4: self.install_s4_rules(datapath)
17     elif dpid == 5: self.install_s5_rules(datapath)
18     elif dpid == 6: self.install_s6_rules(datapath)

```

Listing 12: Configurazione proattiva all'avvio

4.6 Test di Verifica

Per verificare il corretto funzionamento del service slicing, si utilizzano test con `iperf` che misurano la banda effettiva sui due percorsi.

```

1 # Test traffico video (UDP, porta 9999) -> Premium Link
2 # Sul server (H3):
3 H3 iperf -s -u -p 9999 &
4
5 # Sul client (H1):
6 H1 iperf -c 10.0.0.3 -u -p 9999 -b 10M -t 15
7
8 # Risultato atteso: ~5-6 Mbps (limitato dal premium link a 6 Mbps)
9
10 # Test traffico generico (TCP, porta qualsiasi) -> Link Standard
11 # Sul server (H3):
12 H3 iperf -s -p 5001 &
13
14 # Sul client (H1):
15 H1 iperf -c 10.0.0.3 -p 5001 -t 15
16
17 # Risultato atteso: ~1.8-2 Mbps (limitato dal link standard a 2 Mbps)

```

Listing 13: Test del service slicing con iperf

La differenza di circa 3x nella banda misurata conferma che i due tipi di traffico vengono effettivamente instradati su percorsi diversi.

```

mininet> H3 iperf -s -u -p 9999 &
mininet> H3 iperf -s -p 5001 &
-----
Server listening on UDP port 9999
UDP buffer size: 208 KByte (default)
-----
mininet> H1 iperf -c 10.0.0.3 -u -p 9999 -b 10M -t 10
-----
Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IPG target: 1121.52 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 58404 connected with 10.0.0.3 port 9999
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0019 sec 12.5 MBytes 10.5 Mbits/sec
[ 1] Sent 8921 datagrams
[ 3] WARNING: did not receive ack of last datagram after 10 tries.
mininet> H1 iperf -c 10.0.0.3 -p 5001 -t 10
-----
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 56826 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-19.9556 sec 4.00 MBytes 1.68 Mbits/sec
mininet>

```

Figura 6: Test di banda con service slicing: il traffico video (UDP 9999) raggiunge ~ 5.5 Mbps sul Premium Link (con 10 Mbps offerti), mentre il traffico standard (TCP 5001) è limitato a ~ 1.8 Mbps sul percorso standard.

```

*** Starting CLI:
mininet> H3 iperf -s -u -p 9999 &
mininet> H1 iperf -c 10.0.0.3 -u -p 9999 -b 10M -t 15
-----
Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IPG target: 1121.52 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 42766 connected with 10.0.0.3 port 9999
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.0071 sec 18.8 MBytes 10.5 Mbits/sec
[ 1] Sent 13377 datagrams
[ 3] WARNING: did not receive ack of last datagram after 10 tries.

```

Figura 7: Test iperf del traffico video (UDP 9999) sul Premium Link: il client H1 invia a 10 Mbps verso H3; la banda offerta (10.5 Mbps) supera la capacità del Premium Link (6 Mbps), generando packet loss sul link. I 13377 datagrammi inviati in 15 secondi confermano il rate di trasmissione.

```

mininet> H1 iperf -c 10.0.0.3 -p 5001 -t 15
-----
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[  1] local 10.0.0.1 port 56444 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer    Bandwidth
[  1] 0.0000-33.5073 sec  9.13 MBytes  2.28 Mbits/sec
mininet>

```

Figura 8: Test `iperf` del traffico generico (TCP 5001) sul percorso standard: il client H1 raggiunge 2.28 Mbps verso H3, coerente con la capacità del link standard (2 Mbps). Il rapporto tra banda video offerta e banda standard è circa $4.6\times$ ($10.5 / 2.28$).

```

OpenFlow13
cookie=0x0, duration=24.451s, table=0, n_packets=323, n_bytes=13902, priority=100,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=24.451s, table=0, n_packets=197, n_bytes=8610, priority=100,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
cookie=0x0, duration=24.451s, table=0, n_packets=6, n_bytes=476, priority=100,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth3"
cookie=0x0, duration=24.451s, table=0, n_packets=6, n_bytes=476, priority=100,dl_dst=00:00:00:00:00:04 actions=output:"s1-eth4"
cookie=0x0, duration=24.451s, table=0, n_packets=3, n_bytes=126, priority=50,arp,in_port="s1-eth1" actions=output:"s1-eth3",output:"s1-eth4"
cookie=0x0, duration=24.451s, table=0, n_packets=2, n_bytes=84, priority=50,arp,in_port="s1-eth2" actions=output:"s1-eth3",output:"s1-eth4"
cookie=0x0, duration=24.451s, table=0, n_packets=293, n_bytes=12306, priority=50,arp,in_port="s1-eth3" actions=output:"s1-eth1",output:"s1-eth2",output:"s1-eth4"
cookie=0x0, duration=24.451s, table=0, n_packets=293, n_bytes=12306, priority=50,arp,in_port="s1-eth4" actions=output:"s1-eth1",output:"s1-eth2",output:"s1-eth3"
cookie=0x0, duration=24.451s, table=0, n_packets=48, n_bytes=6838, priority=5,ipv6 actions=drop
cookie=0x0, duration=24.451s, table=0, n_packets=0, n_bytes=0, priority=0 actions=CONTROLLER:65535

```

Figura 9: Flow table dello switch S1 con service slicing: le regole a priorità 100 instradano il traffico in base al MAC di destinazione verso le porte corrette. Le regole ARP a priorità 50 gestiscono il broadcast verso tutte le porte di uscita.

5 Dynamic Slicing con Video Preemption

5.1 Concetto di Preemption

Il service slicing instrada staticamente i flussi su percorsi diversi; il dynamic slicing aggiunge un livello di adattamento: il controller monitora il throughput dei Premium Links e modifica le regole di conseguenza.

Il meccanismo centrale è la **video preemption**. Quando i Premium Links sono scarchi, il traffico generico (UDP 800) viene allocato su di essi per usare la banda inutilizzata. Quando però il controller rileva traffico video (UDP 9999), rimuove le regole del traffico generico dai Premium Links, forzandolo a tornare sul percorso standard e liberando banda per il video.

Il termine “preemption” indica che non si tratta di una semplice prioritizzazione (dove il traffico a bassa priorità viene rallentato), ma di una sottrazione esplicita di risorse: le regole dinamiche vengono cancellate dallo switch.

Il sistema identifica tre tipi di traffico tramite la porta UDP di destinazione:

- **Video (UDP 9999)**: sempre instradato sui premium links tramite regole statiche a priorità 200.
- **Dinamico (UDP 800)**: può essere allocato sui premium links tramite regole temporanee a priorità 110, quando la banda è disponibile.
- **Default (qualsiasi altra porta)**: sempre instradato sui percorsi standard tramite regole a priorità 100.

La gerarchia delle priorità garantisce il corretto funzionamento: le regole dinamiche (110) hanno priorità *superiore* a quelle di default (100), quindi quando sono attive dirigo il traffico UDP 800 sul premium link. Al contempo, restano *inferiori* alle regole video (200), che hanno sempre la precedenza. Quando la preemption rimuove le regole dinamiche, il traffico UDP 800 ricade automaticamente sulle regole di default (100) e torna sul percorso standard, senza bisogno di installare regole aggiuntive.

5.2 Stati del Sistema

Il sistema di dynamic slicing opera secondo tre stati distinti, con transizioni governate da soglie configurabili.

Idle Nessun traffico video rilevato. Tutto il traffico, incluso quello generico, può utilizzare liberamente i premium links se le regole lo consentono. Il monitor raccoglie statistiche ma non interviene.

Dynamic Allocation L'utilizzo del premium link è sotto la soglia del 30% e non c'è traffico video attivo. Il controller installa regole a priorità 110 per inoltrare il traffico UDP porta 800 attraverso il premium link, usando la banda altrimenti inutilizzata.

Video Active Traffico video rilevato tramite soglia di throughput (>25% della capacità del premium link). La preemption viene attivata: le regole dinamiche (UDP 800, priorità 110) vengono rimosse, forzando il traffico dinamico a tornare sul percorso standard e liberando il premium link per il video (UDP 9999, priorità 200).

5.3 Soglie di Intervento

Le soglie sono state calibrate empiricamente per bilanciare reattività e stabilità.

Tabella 5: Soglie di intervento per il dynamic slicing

Parametro	Valore	Descrizione
Soglia di banda	30%	Percentuale di utilizzo del premium link sotto la quale il traffico dinamico viene allocato
Soglia di preemption	80%	Percentuale di utilizzo oltre la quale viene attivata la preemption del traffico dinamico
Porta video	UDP 9999	Porta utilizzata per identificare il traffico video
Porta dinamica	UDP 800	Porta utilizzata per il traffico che può essere allocato dinamicamente sui premium links
Intervallo monitor	5 s	Frequenza di raccolta delle statistiche delle porte (port stats)
Intervallo dynamic	2 s	Frequenza di valutazione per allocazione/revoca del traffico dinamico
Smoothing EMA	$\alpha = 0.3$	Fattore della media mobile esponenziale: $B(t) = 0.3 \cdot B_{\text{raw}} + 0.7 \cdot B(t-1)$
Priorità dinamica	110	Priorità delle regole per traffico UDP 800 sui premium links (tra default=100 e video=200)

5.4 Algoritmo di Preemption

I passi dell'algoritmo di preemption sono:

1. **Raccolta statistiche:** ogni 5 secondi (MONITOR_INTERVAL), il monitor interroga tutti gli switch tramite messaggi OpenFlow `OFPPortStatsRequest` per ottenere i contatori TX/RX di ogni porta.
2. **Calcolo della banda:** dalle differenze tra letture successive dei contatori (delta bytes / delta time), il controller calcola il throughput istantaneo in bps per ogni porta di ogni switch.
3. **Smoothing:** per evitare oscillazioni dovute a picchi momentanei, il throughput dei premium links viene filtrato con media mobile esponenziale (EMA):

$$B_{\text{smooth}}(t) = \alpha \cdot B_{\text{raw}}(t) + (1 - \alpha) \cdot B_{\text{smooth}}(t - 1) \quad (1)$$

dove $\alpha = 0.3$. L'EMA è un filtro ricorsivo che considera implicitamente tutta la storia dei campioni, con peso esponenzialmente decrescente. Non opera su una finestra fissa di campioni.

4. **Rilevamento video tramite throughput:** il traffico video (UDP 9999) viene gestito da regole statiche a priorità 200, senza generare packet-in. Il controller rileva la presenza di video monitorando il throughput del premium link: se supera il 25% della capacità (1.5 Mbps su 6 Mbps), il flag `video_active` viene attivato.
5. **Allocazione dinamica:** ogni 2 secondi (`DYNAMIC_CHECK_INTERVAL`), se il premium link è sotto il 30% di utilizzo e non c'è video attivo, il controller installa regole a priorità 110 per inoltrare il traffico UDP porta 800 sul premium link.
6. **Preemption:** quando viene rilevato video o l'utilizzo supera l'80%, il controller rimuove tutte le regole dinamiche (priorità 110), forzando il traffico UDP 800 a tornare sul percorso standard tramite le regole di default (priorità 100). Il contatore `preemption_count` viene incrementato.
7. **Ripristino:** quando il throughput del premium link scende sotto il 5% della capacità, il flag `video_active` viene resettato e il traffico dinamico può essere nuovamente allocato sui premium links.

5.5 Implementazione del Controller (`dynamic_slicing_controller.py`)

Il dynamic slicing controller estende il service slicing con monitoraggio delle porte e logica di allocazione/preemption. Di seguito le funzioni principali con il codice reale.

5.5.1 Inizializzazione e Stato dei Premium Links

Il costruttore definisce soglie, priorità e lo stato di ciascun premium link:

```

1 class DynamicSlicingController(app_manager.RyuApp):
2     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
3     _CONTEXTS = {'wsgi': WSGIApplication}
4
5     def __init__(self, *args, **kwargs):
6         super(DynamicSlicingController, self).__init__(*args, **kwargs)
7
8         # Registrazione WSGI per REST API della dashboard
9         wsgi = kwargs['wsgi']
10        wsgi.register(DashboardAPI, {DASHBOARD_APP_INSTANCE_NAME: self})
11
12        # Costanti: soglie e priorità
13        self.PREMIUM_LINK_CAPACITY_MBPS = 6.0
14        self.BANDWIDTH_THRESHOLD = 0.30      # sotto 30% -> alloca
15        self.PREEMPTION_THRESHOLD = 0.80     # sopra 80% -> preemption
16        self.MONITOR_INTERVAL = 5            # secondi tra port stats
17        self.DYNAMIC_CHECK_INTERVAL = 2      # secondi tra decisioni
18        self.PRIORITY_VIDEO = 200
19        self.PRIORITY_DEFAULT = 100
20        self.PRIORITY_DYNAMIC = 110          # > 100 < 200: sovrascrive
21        default
22        self.PRIORITY_ARP = 50
23
24        # Stato dei link premium (S2 e S4)
25        self.premium_links = {
26            2: {
27                'port': self.PORTS[2]['to_s6_premium'], # Porta 3

```

```

27         'usage_mbps': 0.0,
28         'capacity_mbps': self.PREMIUM_LINK_CAPACITY_MBPS,
29         'dynamic_active': False,
30         'video_active': False
31     },
32     4: {
33         'port': self.PORTS[4]['to_s6_premium'],    # Porta 3
34         'usage_mbps': 0.0,
35         'capacity_mbps': self.PREMIUM_LINK_CAPACITY_MBPS,
36         'dynamic_active': False,
37         'video_active': False
38     }
39 }
40
41 # Variabili per il calcolo del throughput
42 self.prev_tx_bytes = {}    # (dpid, port_no) -> tx_bytes
43 self.prev_timestamp = {}  # (dpid, port_no) -> timestamp
44
45 # Strutture per tracciamento e dashboard
46 self.dynamic_flows = set()
47 self.preemption_count = 0
48 self.dashboard_events = []
49 self.stats_lock = Lock()
50 self.port_stats = {}
51 self.port_speed = {}

```

Listing 14: Inizializzazione del dynamic slicing controller

Il dizionario `premium_links` tiene traccia dell'utilizzo e dello stato di ciascun premium link. I flag `dynamic_active` e `video_active` determinano le transizioni tra gli stati del sistema.

5.5.2 Raccolta Statistiche e Calcolo del Throughput

Il monitor loop interroga tutti gli switch ogni 5 secondi tramite messaggi OpenFlow `OFPPortStatsRequest`:

```

1 def _monitor_loop(self):
2     """Thread in background: raccolta statistiche ogni 5 secondi."""
3     while True:
4         hub.sleep(self.MONITOR_INTERVAL)    # pausa cooperativa (eventlet)
5
6         # Interroga TUTTI gli switch connessi, non solo S2/S4
7         for dpid in list(self.datapaths.keys()):
8             if dpid in self.datapaths:
9                 datapath = self.datapaths[dpid]
10                parser = datapath.ofproto_parser
11                # OFPP_ANY = tutte le porte dello switch
12                req = parser.OFPPortStatsRequest(
13                    datapath, 0, datapath.ofproto.OFPP_ANY)
14                datapath.send_msg(req)    # risposta in
                _port_stats_reply_handler

```

Listing 15: Monitor loop e richiesta statistiche porte

Le risposte vengono elaborate dal gestore `_port_stats_reply_handler`, che calcola il throughput istantaneo da $\text{delta bytes} / \text{delta time}$ e applica lo smoothing EMA sui premium links:

```

1 @set_ev_cls(ofp_event.EventOFPPortStatsReply, MAIN_DISPATCHER)
2 def _port_stats_reply_handler(self, ev):
3     """Gestore risposta statistiche porte: calcola throughput."""
4     body = ev.msg.body          # lista di OFPPortStats
5     dpid = ev.msg.datapath.id    # ID dello switch che risponde
6     current_time = time.time()
7
8     # Porta del link premium per questo switch (None se non ha premium)
9     premium_port = None
10    if dpid in self.premium_links:
11        premium_port = self.premium_links[dpid]['port']
12
13    for stat in body:
14        if stat.port_no > 65000:    # ignora porte speciali (LOCAL, etc.)
15            continue
16        key = (dpid, stat.port_no) # chiave univoca switch+porta
17
18        # Calcolo velocita' da delta bytes / delta time
19        tx_bps = 0.0
20        have_prev = key in self.prev_tx_bytes # serve una lettura
precedente
21        if have_prev:
22            delta_bytes = stat.tx_bytes - self.prev_tx_bytes[key]
23            delta_time = current_time - self.prev_timestamp[key]
24            if delta_time > 0 and delta_bytes >= 0:
25                tx_bps = (delta_bytes * 8.0) / delta_time # byte -> bit
26
27        # Salva valori correnti per la prossima iterazione
28        self.prev_tx_bytes[key] = stat.tx_bytes
29        self.prev_timestamp[key] = current_time
30
31        # Smoothing EMA solo per link premium: B(t) = 0.7*B(t-1) + 0.3*
B_raw
32        # Aggiorna SEMPRE (anche con tx_bps=0) cosi' il valore decade
33        if premium_port and stat.port_no == premium_port and have_prev:
34            throughput_mbps = tx_bps / 1000000.0 # bps -> Mbps
35            old = self.premium_links[dpid]['usage_mbps']
36            self.premium_links[dpid]['usage_mbps'] = \
37                0.7 * old + 0.3 * throughput_mbps # alpha = 0.3

```

Listing 16: Calcolo del throughput con smoothing EMA

Lo smoothing EMA è necessario per evitare che piccole fluttuazioni nel traffico causino continue attivazioni e disattivazioni della preemption. Con $\alpha = 0.3$, il sistema reagisce ai cambiamenti reali del traffico filtrando il rumore.

5.5.3 Logica di Allocazione e Preemption

Il dynamic loop valuta ogni 2 secondi se allocare o revocare il traffico dinamico su ciascun premium link:

```

1 def _dynamic_slicing_loop(self):
2     """Thread in background: decisioni ogni 2 secondi."""
3     while True:
4         hub.sleep(self.DYNAMIC_CHECK_INTERVAL) # 2 secondi
5
6         # Controlla ogni premium link (S2 e S4)

```

```

7         for dpid, link_info in self.premium_links.items():
8             if dpid not in self.datapaths: # switch non ancora
connesso
9                 continue
10
11             # Percentuale di utilizzo corrente (0.0 - 1.0)
12             usage_ratio = link_info['usage_mbps'] / link_info['
capacity_mbps']
13
14             # Rilevamento video: throughput > 25% = 1.5 Mbps su 6 Mbps
15             # Le regole video (prio 200) non generano packet-in,
16             # quindi il video si rileva dal throughput sul premium link
17             if not link_info['video_active'] and usage_ratio > 0.25:
18                 link_info['video_active'] = True
19                 self._add_event('video', 'Flusso video rilevato ...')
20
21             # Reset video: throughput scende sotto 5% = 0.3 Mbps
22             if link_info['video_active'] and usage_ratio < 0.05:
23                 link_info['video_active'] = False
24                 self._add_event('video', 'Flusso video terminato ...')
25
26             # CASO 1 - PREEMPTION: video attivo oppure utilizzo >= 80%
27             # Rimuove le regole a priorit  110 -> traffico UDP 800
28             # ricade sulle regole default (priorit  100 -> standard)
29             if link_info['dynamic_active'] and (
30                 usage_ratio >= self.PREEMPTION_THRESHOLD or
31                 link_info['video_active']
32             ):
33                 self._remove_dynamic_flows(dpid)
34
35             # CASO 2 - ALLOCAZIONE: link scarico (< 30%) e niente video
36             # Installa regole a priorit  110 per UDP 800 su premium
37             elif (not link_info['dynamic_active'] and
38                 usage_ratio < self.BANDWIDTH_THRESHOLD and
39                 not link_info['video_active']):
40                 self._install_dynamic_flows(dpid)

```

Listing 17: Loop di allocazione dinamica e preemption

Il gap tra la soglia di allocazione (30%) e quella di preemption (80%) crea una zona di isteresi che evita oscillazioni continue.

5.5.4 Installazione e Rimozione delle Regole Dinamiche

La funzione `_install_dynamic_flows` installa regole a priorit  110 sia sullo switch con link premium (S2 o S4) sia su S6:

```

1 def _install_dynamic_flows(self, dpid):
2     """Installa regole prio 110 per UDP 800 su premium link."""
3     datapath = self.datapaths[dpid]
4     parser = datapath.ofproto_parser
5     ports = self.PORTS[dpid]
6
7     # Regola sullo switch con link premium (S2 o S4)
8     # Match: IPv4 + UDP + porta destinazione 800
9     match = parser.OFPMatch(
10         in_port=ports['to_s1'], # traffico proveniente da S1
11         eth_type=0x0800, ip_proto=17, # IPv4 + UDP

```

```

12         udp_dst=self.DYNAMIC_UDP_PORT        # porta 800
13     )
14     out_port = ports['to_s6_premium']        # uscita sul link premium
15     actions = [parser.OFPActionOutput(out_port)]
16     # idle_timeout=300: la regola scade dopo 5 min senza pacchetti
17     self._add_flow(datapath, self.PRIORITY_DYNAMIC,
18                   match, actions, idle_timeout=300)
19
20     # Regola corrispondente su S6: serve per instradare il
21     # traffico che arriva dal premium link verso l'host giusto
22     if 6 in self.datapaths:
23         dp_s6 = self.datapaths[6]
24         parser_s6 = dp_s6.ofproto_parser
25         ports_s6 = self.PORTS[6]
26         if dpid == 2:    # percorso superiore
27             s6_in = ports_s6['from_s2_premium']    # porta 2
28             s6_out = ports_s6['H3']                # porta 3 -> H3
29         elif dpid == 4: # percorso inferiore
30             s6_in = ports_s6['from_s4_premium']    # porta 5
31             s6_out = ports_s6['H4']                # porta 6 -> H4
32         match_s6 = parser_s6.OFPMatch(
33             in_port=s6_in, eth_type=0x0800,
34             ip_proto=17, udp_dst=self.DYNAMIC_UDP_PORT)
35         actions_s6 = [parser_s6.OFPActionOutput(s6_out)]
36         self._add_flow(dp_s6, self.PRIORITY_DYNAMIC,
37                       match_s6, actions_s6, idle_timeout=300)
38
39     # Aggiornamento stato: il link ha ora regole dinamiche attive
40     self.premium_links[dpid]['dynamic_active'] = True
41     self.dynamic_flows.add((dpid, self.DYNAMIC_UDP_PORT))

```

Listing 18: Installazione regole dinamiche su switch e S6

La rimozione (`_remove_dynamic_flows`) esegue l'operazione inversa utilizzando `OFPFlowMod` con comando `OFPFC_DELETE_STRICT` per cancellare le regole con lo stesso match e priorità:

```

1 def _remove_dynamic_flows(self, dpid):
2     """Rimuove le regole prio 110: la preemption vera e propria."""
3     datapath = self.datapaths[dpid]
4     parser = datapath.ofproto_parser
5     ports = self.PORTS[dpid]
6
7     # Stesso match usato in _install_dynamic_flows, così' OpenFlow
8     # identifica la regola esatta da cancellare (DELETE_STRICT)
9     match = parser.OFPMatch(
10         in_port=ports['to_s1'],
11         eth_type=0x0800, ip_proto=17,
12         udp_dst=self.DYNAMIC_UDP_PORT    # UDP 800
13     )
14     # _delete_flow invia OFPFlowMod con comando OFPFC_DELETE_STRICT
15     self._delete_flow(datapath, match, self.PRIORITY_DYNAMIC)
16
17     # Rimozione della regola corrispondente su S6
18     if 6 in self.datapaths:
19         dp_s6 = self.datapaths[6]
20         parser_s6 = dp_s6.ofproto_parser
21         ports_s6 = self.PORTS[6]
22         if dpid == 2:    # premium superiore
23             s6_in = ports_s6['from_s2_premium']

```

```

24     elif dpid == 4: # premium inferiore
25         s6_in = ports_s6['from_s4_premium']
26         match_s6 = parser_s6.OFPMatch(
27             in_port=s6_in, eth_type=0x0800,
28             ip_proto=17, udp_dst=self.DYNAMIC_UDP_PORT)
29         self._delete_flow(dp_s6, match_s6, self.PRIORITY_DYNAMIC)
30
31     # Dopo la rimozione, il traffico UDP 800 ricade
32     # automaticamente sulle regole default (prio 100 -> standard)
33     self.premium_links[dpid]['dynamic_active'] = False
34     self.preemption_count += 1 # contatore per la dashboard
35     self._add_event('preemption', 'Preemption eseguita ...')

```

Listing 19: Rimozione regole dinamiche (preemption)

Dopo la rimozione, il traffico UDP 800 viene catturato dalla regola di default (priorità 100) e instradato sul percorso standard, senza dover installare nuove regole.

5.6 Esempio di Scenario

La tabella seguente mostra il ciclo completo di allocazione dinamica e preemption:

Tabella 6: Timeline di un scenario di preemption

Tempo	Evento	Stato del sistema
T = 0s	Avvio sistema	Stato Idle. Nessun traffico attivo, premium links liberi.
T = 2s	Avvio traffico generico	Traffico generico instradato sui percorsi standard. Monitor attivo.
T = 5s	Traffico generico sui premium	Parte del traffico generico finisce sui premium links. Il sistema passa in Dynamic Allocation.
T = 10s	Avvio flusso video UDP 9999	Il controller rileva il video. Stato: Video Active.
T = 12s	Preemption attivata	Traffico generico spostato sui link standard. Premium links riservati al video.
T = 13s	Video a pieno regime	Il flusso video utilizza ~5-6 Mbps sul premium link senza interferenze.
T = 25s	Fine flusso video	Nessun pacchetto video rilevato. Timeout di disattivazione.
T = 28s	Ripristino	Le regole tornano allo stato precedente. Traffico generico riammesso sui premium links. Stato: Idle.

L'intero ciclo — allocazione, preemption, ripristino — avviene senza intervento manuale e senza che gli host debbano essere riconfigurati.

6 Test di Performance

6.1 Metodologia

I test sono stati condotti interamente all'interno dell'ambiente emulato Mininet, utilizzando i seguenti strumenti:

- **iperf**: per la generazione di traffico TCP e UDP e la misurazione della banda effettiva.
- **ping**: per la misurazione della latenza e della raggiungibilità.
- **Mininet CLI**: per i comandi `pingall` e l'interazione diretta con gli host emulati.
- **tcpdump**: per la cattura e l'analisi dei pacchetti su interfacce specifiche.

I valori di banda riportati nelle tabelle successive sono il risultato di più esecuzioni per ciascuno scenario di test.

6.2 Risultati Topology Slicing

Con il topology slicing attivo, il comando `pingall` conferma l'isolamento tra le due slice: solo 4 coppie su 12 comunicano (drop rate 66.7%).

```

1 mininet> pingall
2 *** Ping: testing ping reachability
3 H1 -> X H3 X
4 H2 -> X X H4
5 H3 -> H1 X X
6 H4 -> X H2 X
7 *** Results: 66% dropped (4/12 received)

```

Listing 20: Risultati del `pingall` con topology slicing

Le coppie comunicanti (H1–H3, H2–H4) corrispondono a quelle previste dal design delle slice. I ping intra-slice confermano la connettività con RTT coerente con il percorso standard (2×50 ms di latenza sui link S2–S3 e S3–S6):

Tabella 7: Risultati dei test ping con topology slicing

Test	Packet Loss	RTT avg	RTT mdev	Esito
H1 → H3 (intra-slice)	0%	283 ms	79 ms	Connettività OK
H2 → H4 (intra-slice)	0%	250 ms	10 ms	Connettività OK
H1 → H4 (cross-slice)	100%	—	—	Bloccato (Dest. Unreachable)
H2 → H3 (cross-slice)	100%	—	—	Bloccato (Dest. Unreachable)

I ping cross-slice restituiscono **Destination Host Unreachable** perché il controller scarta silenziosamente i pacchetti tra slice diverse (drop nel `packet_in_handler`). Le risposte ARP della destinazione non vengono mai generate, dato che il broadcast è confinato alla slice della sorgente.

```
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ sudo ovs-ofctl dump-flows s1 --protocols=OpenFlow13
cookie=0x0, duration=302.938s, table=0, n_packets=9, n_bytes=714, idle_timeout=300, priority=10,dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=302.930s, table=0, n_packets=8, n_bytes=616, idle_timeout=300, priority=10,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth3"
cookie=0x0, duration=293.142s, table=0, n_packets=3, n_bytes=238, idle_timeout=300, priority=10,dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
cookie=0x0, duration=293.132s, table=0, n_packets=2, n_bytes=140, idle_timeout=300, priority=10,dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04 actions=output:"s1-eth4"
cookie=0x0, duration=353.410s, table=0, n_packets=89, n_bytes=10966, priority=5,ipv6 actions=drop
cookie=0x0, duration=353.410s, table=0, n_packets=36, n_bytes=1624, priority=0 actions=CONTROLLER:65535
```

Figura 10: Flow table dello switch S1 con topology slicing: le regole instradano il traffico verso S2 (upper slice) o S4 (lower slice) in base al MAC di destinazione.

6.3 Risultati Service Slicing

Con il service slicing attivo, il `pingall` restituisce 0% drop (12/12 connessioni), confermando la connettività completa.

I test di banda con `iperf` hanno confermato la differenziazione tra traffico video e traffico generico:

Tabella 8: Risultati dei test di banda con service slicing

Tipo Traffico	Percorso	Banda Link	Banda Offerta	Banda Ricevuta
Video (UDP 9999)	Premium (S2–S6)	6 Mbps	10 Mbps	~5.5 Mbps
Generico (TCP 5001)	Standard (S2–S3–S6)	2 Mbps	max	1.81 Mbps
Video (UDP 9999)	Premium (S4–S6)	6 Mbps	10 Mbps	~5.5 Mbps
Generico (TCP 5001)	Standard (S4–S5–S6)	2 Mbps	max	1.77 Mbps

La banda video offerta (10 Mbps) supera la capacità del Premium Link (6 Mbps); il valore ricevuto lato server (~5.5 Mbps) riflette il collo di bottiglia.

Il rapporto tra la banda ricevuta video e quella standard è di circa $3\times$ ($\sim 5.5 / 1.81$), coerente con il rapporto tra le capacità dei link (6 vs 2 Mbps).

La latenza misurata con `ping` tra H1 e H3 (percorso standard, poiché ICMP non è UDP 9999) è:

```
1 --- 10.0.0.3 ping statistics ---
2 10 packets transmitted, 10 received, 0% packet loss, time 9123ms
3 rtt min/avg/max/mdev = 230.258/244.119/263.620/9.259 ms
```

Listing 21: RTT misurato H1→H3 via percorso standard

L'RTT medio di 244 ms è coerente con il percorso standard (S1→S2→S3→S6): due link da 50 ms $\times$ 2 (andata e ritorno) = 200 ms, più l'overhead di propagazione e switching. Il basso mdev (9.3 ms) indica stabilità del percorso.

```
mininet> pingall
*** Ping: testing ping reachability
H1 -> H2 H3 H4
H2 -> H1 H3 H4
H3 -> H1 H2 H4
H4 -> H1 H2 H3
*** Results: 0% dropped (12/12 received)
mininet> █
```

Figura 11: Risultato del `pingall` con service slicing: tutti gli host comunicano tra loro (0% packet loss), a differenza del topology slicing.

```
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ sudo ovs-ofctl dump-flows s2 --protocols=OpenFlow13
    cookie=0x0, duration=220.030s, table=0, n_packets=0, n_bytes=0, priority=200,udp,dl_dst=00:00:00:00:00:03,tp_dst=9999 actions=output:"s2-eth3"
    cookie=0x0, duration=220.030s, table=0, n_packets=2048, n_bytes=86240, priority=100,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth1"
    cookie=0x0, duration=220.030s, table=0, n_packets=6, n_bytes=476, priority=100,dl_dst=00:00:00:00:00:03 actions=output:"s2-eth2"
    cookie=0x0, duration=220.030s, table=0, n_packets=3050, n_bytes=128100, priority=50,arp,in_port="s2-eth1" actions=output:"s2-eth2"
    cookie=0x0, duration=220.030s, table=0, n_packets=3045, n_bytes=127890, priority=50,arp,in_port="s2-eth2" actions=output:"s2-eth1"
    cookie=0x0, duration=220.030s, table=0, n_packets=97, n_bytes=15252, priority=5,ipv6 actions=drop
    cookie=0x0, duration=220.030s, table=0, n_packets=0, n_bytes=0, priority=0 actions=CONTROLLER:65535
```

Figura 12: Flow table di S2 con service slicing: la regola a priorità 200 per il traffico video (UDP 9999) dirige i pacchetti sulla porta 3 (Premium Link), mentre la regola a priorità 100 instrada il traffico generico sulla porta 2 (percorso standard).

6.4 Stress Test

Per verificare l'isolamento effettivo del Premium Link sotto carico, sono stati eseguiti due scenari di stress test utilizzando il `service_slicing_controller.py` con traffico simultaneo video, TCP e ping tra H1 e H3.

Scenario A – Video + TCP + Ping simultanei. Tre flussi concorrenti: traffico video UDP sulla porta 9999 a 4 Mbps (instradato sul Premium Link), traffico TCP sulla porta 5001 (instradato sul percorso standard S2→S3→S6) e un ping ICMP (anch'esso sul percorso standard). L'obiettivo è verificare se la congestione TCP influenza il traffico video.

Scenario B – Solo Video + Ping (senza TCP). Solo traffico video UDP 9999 a 4 Mbps e ping ICMP, senza il flusso TCP. Serve come baseline per confrontare l'RTT del ping in assenza di congestione.

```
1 # === SCENARIO A: Video + TCP + Ping simultanei ===
2 # Avvio server iperf su H3 (video UDP e TCP)
3 H3 iperf -s -u -p 9999 &
4 H3 iperf -s -p 5001 &
5
```

```

6 # Avvio traffico video UDP 9999 a 4 Mbps (20 s, Premium Link)
7 H1 iperf -c 10.0.0.3 -u -p 9999 -b 4M -t 20 > /tmp/stress_video_a.txt &
8
9 # Avvio traffico TCP sulla porta 5001 (18 s, percorso standard)
10 H1 iperf -c 10.0.0.3 -p 5001 -t 18 > /tmp/stress_tcp_a.txt &
11
12 # Ping ICMP (15 pacchetti, percorso standard)
13 H1 ping -c 15 10.0.0.3
14
15 # === SCENARIO B: Solo Video + Ping (senza TCP) ===
16 # Avvio server iperf su H3 (solo video UDP)
17 H3 iperf -s -u -p 9999 &
18
19 # Avvio traffico video UDP 9999 a 4 Mbps (20 s, Premium Link)
20 H1 iperf -c 10.0.0.3 -u -p 9999 -b 4M -t 20 > /tmp/stress_video_b.txt &
21
22 # Ping ICMP (15 pacchetti, percorso standard senza congestione TCP)
23 H1 ping -c 15 10.0.0.3

```

Listing 22: Comandi eseguiti per lo stress test (Scenario A e B)

Tabella 9: Confronto Stress Test: Scenario A (Video+TCP+Ping) vs Scenario B (Video+Ping)

Metrica	Scenario A	Scenario B
Banda video (UDP 9999)	4.20 Mbps	4.20 Mbps
Packet loss video	0%	0%
Jitter video	2.318 ms	1.985 ms
RTT ping medio	1714 ms	264 ms*
RTT ping mdev	995 ms	30 ms*
Packet loss ping	0%	0%

* Valori a regime (ultimi 10 campioni); i primi ping dello Scenario B presentano RTT elevato residuo dal buffer dello Scenario A.

I risultati confermano due proprietà del service slicing:

1. **Isolamento del Premium Link:** la banda video è identica in entrambi gli scenari (4.20 Mbps, 0% loss), dimostrando che il traffico TCP sul percorso standard non impatta il link dedicato al video.
2. **Congestione del percorso standard:** l'RTT del ping nello Scenario A (1714 ms) è circa 6× superiore allo Scenario B (264 ms), poiché il traffico TCP satura il percorso standard (S2→S3→S6) e i pacchetti ICMP subiscono accodamento.

```

PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=1823 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=1632 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=1723 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=1839 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=2282 ms
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-20.0050 sec 10.0 MBytes 4.19 Mbits/sec
[ 1] Sent 7137 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-20.6970 sec 10.0 MBytes 4.05 Mbits/sec 2.372 ms 0/7136 (0%)
[ 1] 0.0000-20.6970 sec 3 datagrams received out-of-order
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=2433 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=2625 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=2868 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=4311 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=4797 ms
64 bytes from 10.0.0.3: icmp_seq=11 ttl=64 time=5486 ms
64 bytes from 10.0.0.3: icmp_seq=12 ttl=64 time=5822 ms
64 bytes from 10.0.0.3: icmp_seq=13 ttl=64 time=6290 ms
64 bytes from 10.0.0.3: icmp_seq=14 ttl=64 time=6647 ms
64 bytes from 10.0.0.3: icmp_seq=15 ttl=64 time=7062 ms

--- 10.0.0.3 ping statistics ---
15 packets transmitted, 15 received, 0% packet loss, time 14181ms
rtt min/avg/max/mdev = 1631.946/3842.811/7062.150/1937.238 ms, pipe 5
mininet>

```

Figura 13: Stress test Scenario A: tre flussi simultanei H1→H3. Il traffico video sul Premium Link non è impattato (4.20 Mbps), mentre il ping sul percorso standard subisce latenza elevata per la congestione TCP.

```

PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=252 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=246 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=263 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=237 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=248 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=246 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=224 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=245 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=226 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=232 ms
64 bytes from 10.0.0.3: icmp_seq=11 ttl=64 time=248 ms
64 bytes from 10.0.0.3: icmp_seq=12 ttl=64 time=231 ms
64 bytes from 10.0.0.3: icmp_seq=13 ttl=64 time=226 ms
64 bytes from 10.0.0.3: icmp_seq=14 ttl=64 time=234 ms
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-20.0056 sec 10.0 MBytes 4.19 Mbits/sec
[ 1] Sent 7137 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-20.0160 sec 10.0 MBytes 4.19 Mbits/sec 2.326 ms 0/7136 (0%)
[ 1] 0.0000-20.0160 sec 5 datagrams received out-of-order
64 bytes from 10.0.0.3: icmp_seq=15 ttl=64 time=269 ms

--- 10.0.0.3 ping statistics ---
15 packets transmitted, 15 received, 0% packet loss, time 14050ms
rtt min/avg/max/mdev = 224.058/241.790/269.009/13.024 ms
mininet>

```

Figura 14: Stress test Scenario B: senza traffico TCP, il ping mostra RTT stabile (~264 ms), confermando che la congestione dello Scenario A è causata dal TCP sul percorso standard.

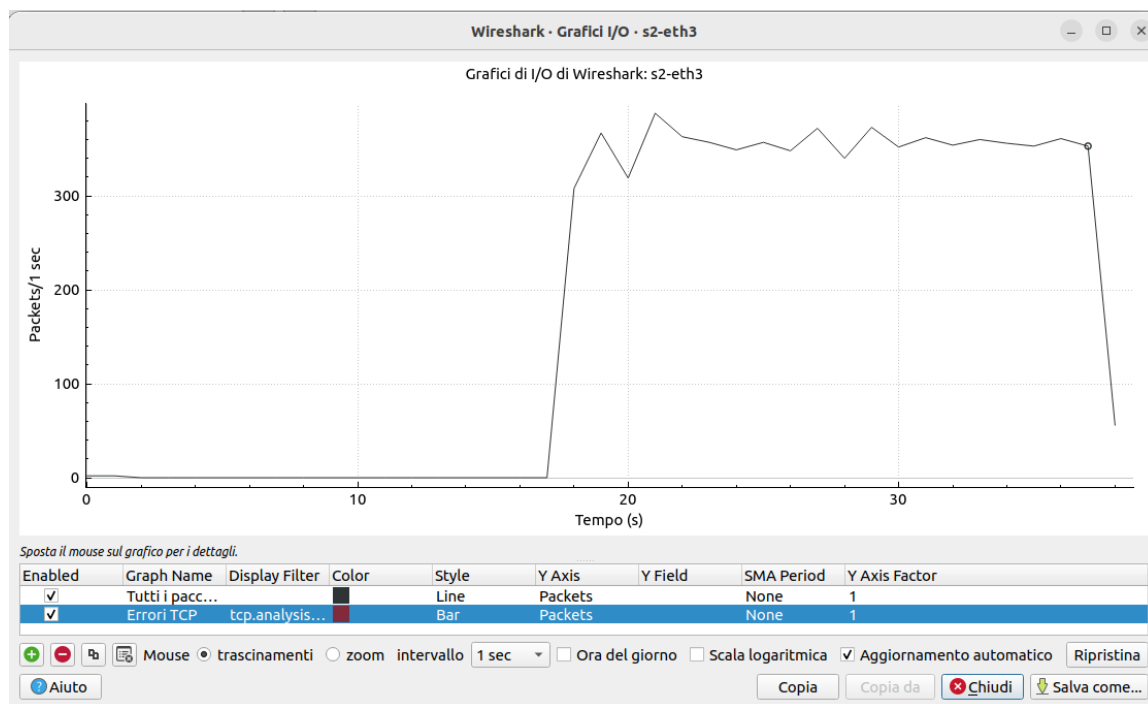


Figura 15: Grafico I/O di Wireshark durante lo stress test sul Premium Link (interfaccia s2-eth3): il packet rate sale a ~350 pacchetti/s quando il traffico video viene attivato e rimane stabile per tutta la durata del flusso, confermando che il Premium Link gestisce il video senza congestione.

6.5 Risultati Dynamic Slicing

Il test del dynamic slicing ha verificato il ciclo completo di allocazione dinamica e pre-emption. Con pingall: 0% drop (12/12).

I comandi eseguiti nella CLI di Mininet per il test completo:

```

1 # 1. Avvio del server iperf su H3 (riceve sia UDP 800 che UDP 9999)
2 H3 iperf -s -u -p 800 &
3 H3 iperf -s -u -p 9999 &
4
5 # 2. Avvio del traffico dinamico (UDP porta 800, 1 Mbps, 60 secondi)
6 #   Il controller allocherà questo flusso sul Premium Link (regola
7   prio 110)
8 H1 iperf -c 10.0.0.3 -u -p 800 -b 1M -t 60 &
9
10 # 3. Attesa 15 secondi per la stabilizzazione
11 #   Verifica: dump-flows s2 mostra la regola priority=110
12 sudo ovs-ofctl dump-flows s2 --protocols=OpenFlow13
13
14 # 4. Verifica via REST API: dynamic_active = true
15 curl -s http://127.0.0.1:8080/api/stats | python3 -m json.tool
16
17 # 5. Avvio del flusso video (UDP porta 9999, 4 Mbps, 30 secondi)
18 #   Il controller esegue la preemption: rimuove la regola prio 110
19 H1 iperf -c 10.0.0.3 -u -p 9999 -b 4M -t 30
20
21 # 6. Durante il video, verifica: la regola prio 110 e' sparita
22 sudo ovs-ofctl dump-flows s2 --protocols=OpenFlow13

```

```

23 # 7. Dopo la fine del video (~20 s), la regola prio 110 viene
    reinstallata

```

Listing 23: Comandi eseguiti per il test di dynamic slicing

La procedura e i risultati sono riportati nella tabella seguente:

Tabella 10: Risultati del test di allocazione dinamica e preemption

Fase	Azione	Verifica	Esito
1	Avvio UDP 800 a 1 Mbps	Attesa 15 s	Traffico instradato
2	dump-flows s2	Cerca priority=110	Presente
3	API /api/stats	dynamic_active	True
4	Avvio video UDP 9999 a 4 Mbps	Attesa 10 s	Preemption triggerata
5	dump-flows s2	Cerca priority=110	Rimossa
6	API /api/stats	video_active	True
7	API /api/stats	preemption_count	1
8	Fine video, attesa 20 s	dump-flows s2	Ripristino regole

I dump della flow table di S2 nei tre momenti chiave confermano il funzionamento:

```

1 # sudo ovs-ofctl dump-flows s2 --protocols=OpenFlow13
2 priority=200,udp,tp_dst=9999          actions=output:3    # video ->
   premium
3 priority=100,ip,in_port=1             actions=output:2    # default ->
   standard
4 priority=110,udp,in_port=1,tp_dst=800 actions=output:3    # DINAMICA ->
   premium
5 priority=50,arp,in_port=1             actions=output:2    # ARP
6 priority=0                            actions=CONTROLLER  # table-miss

```

Listing 24: Flow table S2 durante allocazione dinamica (regola prio 110 presente)

```

1 # sudo ovs-ofctl dump-flows s2 --protocols=OpenFlow13
2 priority=200,udp,tp_dst=9999          actions=output:3    # video (
   n_packets=3445)
3 priority=100,ip,in_port=1             actions=output:2    # default ->
   standard
4 # La regola priority=110 e' stata RIMOSSA dalla preemption
5 # UDP 800 ricade sulla regola priority=100 -> percorso standard
6 priority=50,arp,in_port=1             actions=output:2    # ARP
7 priority=0                            actions=CONTROLLER  # table-miss

```

Listing 25: Flow table S2 durante preemption (regola prio 110 rimossa)

La REST API (/api/stats) durante il video ha restituito: usage_mbps: 2.04, video_active: true, dynamic_active: false, preemption_count: 1. Il log del controller contiene i messaggi di allocazione, rilevamento video e preemption, confermando il corretto funzionamento dell'intero ciclo.

```

>>> ALLOCAZIONE DINAMICA su S2: utilizzo=0.0% (sotto soglia 30%) <<<
Regola dinamica installata su S2: UDP 800 -> porta 3 (premium, idle_timeout=300s)
Regola dinamica installata su S6: da porta 2 -> porta 3 (idle_timeout=300s)
Allocazione dinamica COMPLETATA su S2

```

Figura 16: Log del controller: allocazione dinamica attivata su S2. Il Premium Link è scarico (utilizzo 0%), le regole a priorità 110 per UDP 800 vengono installate su S2 e S6.

```
Video rilevato su S2 tramite throughput: 1.9 Mbps
*** PREEMPTION su S2: utilizzo=30.8%, video=True ***
Regola rimossa su S2 (priorita' 110)
Regola dinamica RIMOSSA da S2
Regola rimossa su S6 (priorita' 110)
Regola dinamica RIMOSSA da S6 (porta ingresso 2)
Preemption COMPLETATA su S2 - traffico dinamico rimosso
```

Figura 17: Log del controller: preemption attivata su S2. Il traffico video è stato rilevato (video=True) e le regole dinamiche a priorità 110 vengono rimosse da S2 e S6 per liberare il Premium Link.

```
>>> ALLOCAZIONE DINAMICA su S2: utilizzo=4.6% (sotto soglia 30%) <<<
Regola dinamica installata su S2: UDP 800 -> porta 3 (premium, idle_timeout=300s)
Regola dinamica installata su S6: da porta 2 -> porta 3 (idle_timeout=300s)
Allocazione dinamica COMPLETATA su S2
```

Figura 18: Log del controller: ripristino dopo la fine del video. Il throughput è tornato sotto la soglia del 30% e le regole dinamiche vengono reinstallate, completando il ciclo di preemption.

```
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ sudo ovs-ofctl
dump-flows s2 --protocols=OpenFlow13
cookie=0x0, duration=118.161s, table=0, n_packets=0, n_bytes=0, priority=200,udp,
tp_dst=9999 actions=output:"s2-eth3"
cookie=0x0, duration=116.198s, table=0, n_packets=7637, n_bytes=11547144, idle_
timeout=300, priority=110,udp,in_port="s2-eth1",tp_dst=800 actions=output:"s2-et
h3"
cookie=0x0, duration=118.161s, table=0, n_packets=0, n_bytes=0, priority=100,ip
,in_port="s2-eth1" actions=output:"s2-eth2"
cookie=0x0, duration=118.161s, table=0, n_packets=391, n_bytes=16422, priority=
50,arp,in_port="s2-eth1" actions=output:"s2-eth2"
cookie=0x0, duration=118.161s, table=0, n_packets=0, n_bytes=0, priority=50,arp
,in_port="s2-eth2" actions=output:"s2-eth1"
cookie=0x0, duration=118.161s, table=0, n_packets=1214, n_bytes=54798, priority
=100,in_port="s2-eth2" actions=output:"s2-eth1"
cookie=0x0, duration=118.161s, table=0, n_packets=22, n_bytes=2829, priority=10
0,in_port="s2-eth3" actions=output:"s2-eth1"
cookie=0x0, duration=118.161s, table=0, n_packets=22, n_bytes=2829, priority=0
actions=CONTROLLER:65535
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$
```

Figura 19: Flow table di S2 durante l'allocazione dinamica: la regola a priorità 110 per UDP 800 instrada il traffico generico sul Premium Link (output:s2-eth3).

```
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ sudo ovs-ofctl dump-flows s2 --protocols=
OpenFlow13
cookie=0x0, duration=227.126s, table=0, n_packets=8533, n_bytes=12901896, priority=200,udp,tp_dst=9999 acti
ons=output:"s2-eth3"
cookie=0x0, duration=227.126s, table=0, n_packets=10735, n_bytes=16217780, priority=100,ip,in_port="s2-eth1
" actions=output:"s2-eth2"
cookie=0x0, duration=227.126s, table=0, n_packets=5756, n_bytes=241752, priority=50,arp,in_port="s2-eth1" a
ctions=output:"s2-eth2"
cookie=0x0, duration=227.126s, table=0, n_packets=0, n_bytes=0, priority=50,arp,in_port="s2-eth2" actions=o
utput:"s2-eth1"
cookie=0x0, duration=227.126s, table=0, n_packets=9689, n_bytes=415126, priority=100,in_port="s2-eth2" acti
ons=output:"s2-eth1"
cookie=0x0, duration=227.126s, table=0, n_packets=25, n_bytes=3882, priority=100,in_port="s2-eth3" actions=
output:"s2-eth1"
cookie=0x0, duration=227.126s, table=0, n_packets=24, n_bytes=3792, priority=0 actions=CONTROLLER:65535
```

Figura 20: Flow table di S2 dopo la preemption: la regola a priorità 110 è stata rimossa e il traffico UDP 800 ricade sulla regola di default (priorità 100, percorso standard). Il contatore della regola video (priority=200) conferma il traffico attivo.

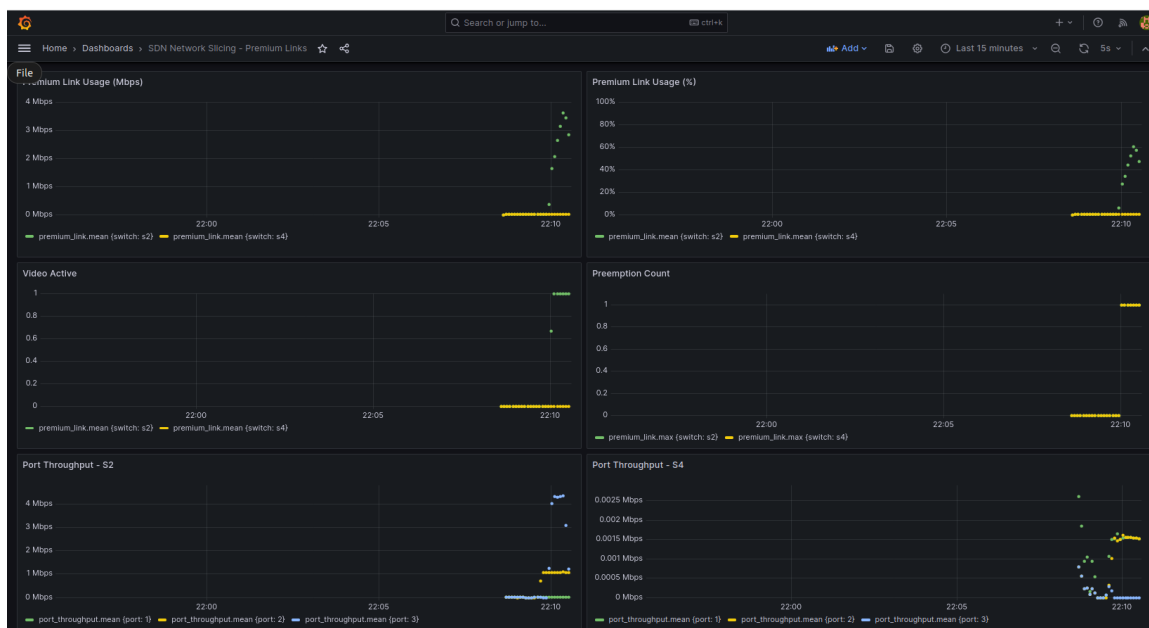


Figura 21: Dashboard Grafana durante un evento di preemption: il pannello “Premium Link Usage” mostra il picco a ~3–4 Mbps sul Premium Link di S2, “Video Active” passa a 1 e “Preemption Count” viene incrementato. Il pannello “Port Throughput – S2” mostra il traffico concentrato sulla porta 3 (Premium Link verso S6).

```
1 $ curl -s http://127.0.0.1:8080/api/stats | python3 -m json.tool
2 {
3   "premium_links": {
4     "upper": {
5       "usage_mbps": 0.0,
6       "capacity_mbps": 6.0,
7       "dynamic_active": true,           # allocazione attiva
8       "video_active": false
9     },
10    "lower": {
11      "usage_mbps": 0.0,
12      "capacity_mbps": 6.0,
13      "dynamic_active": true,
14      "video_active": false
```

```

15     }
16 },
17 "preemption_count": 0,
18 "events": [
19   {"timestamp": "22:08:33", "type": "dynamic",
20    "message": "Allocazione dinamica attivata sul percorso superiore (
S2 -> S6)"},
21   {"timestamp": "22:08:33", "type": "dynamic",
22    "message": "Allocazione dinamica attivata sul percorso inferiore (
S4 -> S6)"}
23 ]
24 }

```

Listing 26: REST API /api/stats — stato dopo l'allocazione dinamica

```

1 $ curl -s http://127.0.0.1:8080/api/stats | python3 -m json.tool
2 {
3   "premium_links": {
4     "upper": {
5       "usage_mbps": 3.376,           # traffico video attivo
6       "capacity_mbps": 6.0,
7       "dynamic_active": false,      # allocazione rimossa
8       "video_active": true          # video rilevato
9     },
10    "lower": {
11      "usage_mbps": 0.0,
12      "capacity_mbps": 6.0,
13      "dynamic_active": true,
14      "video_active": false
15    }
16  },
17  "preemption_count": 2,             # 2 preemption eseguite
18  "events": [
19    {"timestamp": "22:11:41", "type": "preemption",
20     "message": "Preemption eseguita sul percorso superiore (S2 -> S6)"
21   },
22    {"timestamp": "22:11:41", "type": "video",
23     "message": "Flusso video rilevato sul percorso superiore (1.6 Mbps)
24   },
25    {"timestamp": "22:11:01", "type": "dynamic",
26     "message": "Allocazione dinamica attivata sul percorso superiore (
S2 -> S6)"},
27    {"timestamp": "22:10:01", "type": "preemption",
28     "message": "Preemption eseguita sul percorso superiore (S2 -> S6)"}
29  ]
30 }

```

Listing 27: REST API /api/stats — stato durante la preemption (video attivo)

Il confronto tra i due snapshot della REST API mostra il ciclo completo: nella prima query `dynamic_active` è `true` su entrambi i percorsi e il contatore di preemption è a zero; nella seconda query, dopo l'arrivo del video, `video_active` passa a `true` sul percorso superiore, `dynamic_active` diventa `false` (il traffico dinamico è stato rimosso) e il contatore di preemption sale a 2.

6.6 Verifica delle Flow Table

Oltre ai test di banda, il corretto funzionamento del sistema può essere verificato ispezionando direttamente le regole installate sugli switch tramite il comando `ovs-ofctl`, controllando che le regole OpenFlow corrispondano al comportamento atteso.

```

1 # sudo ovs-ofctl dump-flows s2 --protocols=OpenFlow13
2 priority=200,udp,d1_dst=00:...:03,tp_dst=9999
3   actions=output:3      # 9116 pacchetti video -> premium
4 priority=100,d1_dst=00:...:01
5   actions=output:1      # 2991 pacchetti ritorno verso H1
6 priority=100,d1_dst=00:...:03
7   actions=output:2      # 1692 pacchetti standard -> S3
8 priority=50,arp,in_port=1 actions=output:2 # ARP forward
9 priority=50,arp,in_port=2 actions=output:1 # ARP reverse
10 priority=5,ipv6          actions=drop      # scarta IPv6
11 priority=0               actions=CONTROLLER # table-miss

```

Listing 28: Regole reali su S2 con service slicing (output misurato)

La presenza della regola a priorità 200 con match su `udp_dst=9999` e azione `output:3` conferma che il traffico video viene instradato sulla porta 3 di S2 (il Premium Link verso S6). La regola a priorità 100 con `output:2` gestisce il traffico generico verso il percorso standard (S3).

Con il dynamic slicing attivo, durante l'allocazione dinamica compaiono anche regole a priorità 110 per il traffico UDP porta 800:

```

1 # Regola dinamica (presente solo quando allocazione attiva)
2 # priority=110, udp, udp_dst=800 -> output:3 (premium link)
3
4 # Dopo la preemption, questa regola viene rimossa e il traffico
5 # UDP 800 ricade sulla regola default (priority=100) -> output:2

```

Listing 29: Regole dinamiche visibili durante l'allocazione

Il contatore `n_packets` di ciascuna regola permette inoltre di verificare che i pacchetti vengano effettivamente catturati dalla regola corretta. Ad esempio, durante un test video si osserva il contatore della regola a priorità 200 crescere, confermando che i pacchetti UDP 9999 seguono il percorso premium.

Analogamente, si può utilizzare `tcpdump` sulle interfacce degli switch per verificare il percorso fisico dei pacchetti:

```

1 # Verifica che il traffico video passi per il premium link (S2 porta 3)
2 sudo tcpdump -i s2-eth3 -n udp port 9999
3
4 # Verifica che il traffico generico passi per il percorso standard (S2
5 # porta 2)
6 sudo tcpdump -i s2-eth2 -n

```

Listing 30: Verifica del percorso con tcpdump

```

23:32:19.258459 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.374736 ARP, Request who-has 10.0.0.4 tell 10
P, length 1470 .0.0.3, length 28
23:32:19.259134 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.374738 ARP, Request who-has 10.0.0.4 tell 10
P, length 1470 .0.0.2, length 28
23:32:19.291112 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.374740 ARP, Request who-has 10.0.0.2 tell 10
P, length 1470 .0.0.1, length 28
23:32:19.291130 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.374741 ARP, Request who-has 10.0.0.3 tell 10
P, length 1470 .0.0.1, length 28
23:32:19.304676 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.374743 ARP, Request who-has 10.0.0.4 tell 10
P, length 1470 .0.0.1, length 28
23:32:19.311748 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.442780 ARP, Request who-has 10.0.0.3 tell 10
P, length 1470 .0.0.2, length 28
23:32:19.326766 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.442816 ARP, Request who-has 10.0.0.4 tell 10
P, length 1470 .0.0.3, length 28
23:32:19.336699 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.442822 ARP, Request who-has 10.0.0.4 tell 10
P, length 1470 .0.0.2, length 28
23:32:19.346228 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.442827 ARP, Request who-has 10.0.0.2 tell 10
P, length 1470 .0.0.1, length 28
23:32:19.359466 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.442833 ARP, Request who-has 10.0.0.3 tell 10
P, length 1470 .0.0.1, length 28
23:32:19.370222 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.442837 ARP, Request who-has 10.0.0.4 tell 10
P, length 1470 .0.0.1, length 28
23:32:19.380575 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.462890 ARP, Reply 10.0.0.3 is-at 00:00:00:00
P, length 1470 :00:03, length 28
23:32:19.389748 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.462904 ARP, Reply 10.0.0.4 is-at 00:00:00:00
P, length 1470 :00:04, length 28
23:32:19.400519 IP 10.0.0.1.54477 > 10.0.0.3.9999: UD 23:32:30.546175 ARP, Reply 10.0.0.3 is-at 00:00:00:00
P, length 1470 :00:03, length 28
23:32:30.546185 ARP, Reply 10.0.0.4 is-at 00:00:00:00

```

Figura 22: Verifica del percorso con `tcpdump`: il traffico video (UDP 9999) transita su `s2-eth3` (Premium Link verso S6), mentre il traffico generico passa per `s2-eth2` (percorso standard verso S3).

6.7 Analisi dei Pacchetti con Wireshark

Per un'analisi approfondita del traffico a livello di pacchetto, sono state effettuate catture Wireshark sulle interfacce degli switch durante i test di service slicing e dynamic slicing. Le catture forniscono una verifica indipendente rispetto ai contatori OpenFlow: mentre `ovs-ofctl dump-flows` mostra i contatori aggregati delle regole, Wireshark permette di ispezionare ogni singolo pacchetto e confermare che il tipo di traffico corrisponda al percorso osservato.

Traffico video sul Premium Link. La cattura sulla porta premium di S2 (`s2-eth3`) durante un flusso video UDP 9999 conferma che i pacchetti vengono correttamente instradati sul Premium Link. La Figura 23 mostra il flusso di pacchetti UDP da 10.0.0.1 (H1) verso 10.0.0.3 (H3) sulla porta 9999, con dimensione costante di 1512 byte per pacchetto e inter-arrival time regolare, caratteristico di un flusso video a bitrate costante.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...h...z1...E...HI...'
2	0.002568	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...h...z1...HI...'
3	0.002572	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...j...z1...HI...'
4	0.005399	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...m...z1...HI...'
5	0.008199	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...l...z1...HI...'
6	0.010904	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...o...z1...HI...'
7	0.013922	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...n...z1...HI...'
8	0.016686	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...a...z1...HI...'
9	0.020061	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...g...z1...HI...'
10	0.024697	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...c...z1...EB...HI...'
11	0.029462	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...b...z1...U...HI...'
12	0.030203	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...e...z1...HI...'
13	0.031379	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...d...z1...HI...'
14	0.034439	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...g...z1...HI...'
15	0.036135	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...f...z1...HI...'
16	0.038851	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...y...z1...HI...'
17	0.041690	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...x...z1...F...HI...'
18	0.044066	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...z...z1...HI...'
19	0.047284	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...z...z1...HI...'
20	0.050389	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...j...z1...HI...'
21	0.053346	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...j...z1...HI...'
22	0.055734	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...j...z1...HI...'
23	0.059611	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...q...z1...HI...'
24	0.061674	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...q...z1...HI...'
25	0.064023	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...p...z1...X...HI...'
26	0.067084	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...s...z1...HI...'
27	0.069681	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...r...z1...F...HI...'
28	0.072535	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...u...z1...HI...'
29	0.075434	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...t...z1...HI...'
30	0.078489	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...w...z1...HI...'
31	0.084962	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...v...z1...LF...HI...'
32	0.088972	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...i...z1...HI...'
33	0.088975	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...h...z1...B...HI...'
34	0.093992	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...K...z1...H...HI...'
35	0.094255	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...J...z1...V...HI...'
36	0.095917	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...SM...z1...bc...HI...'
37	0.097761	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...SL...z1...HI...'
38	0.100534	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...GO...z1...td...HI...'
39	0.103326	10.0.0.1	10.0.0.3	TPLINK	1512	UDP Cmd: ...N...z1...HT...'

Frame 1: 1512 bytes on wire (12096 bits), 1512 bytes captured (12096 bits) on interface eth2
 Ethernet II, Src: 00:00:00:00:00:01 (00:00:00:00:00:01), Dst: 00:00:00:00:00:03 (00:00:00:00:00:03)
 Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.3
 User Datagram Protocol, Src Port: 47137, Dst Port: 9999
 TP-Link Smart Home Protocol
 Frame (1512 bytes) JSRV message (1410 bytes)

Figura 23: Cattura Wireshark sul Premium Link (S2→S6): flusso video UDP 9999 da H1 (10.0.0.1) a H3 (10.0.0.3). I pacchetti hanno dimensione uniforme (1512 byte) e inter-arrival time regolare (~2.5 ms), confermando l'instradamento sul percorso premium a 6 Mbps.

Traffico generico sul percorso standard. La cattura sulla porta standard di S2 (s2-eth2) mostra il traffico TCP sulla porta 5001, instradato correttamente sul percorso standard (S2→S3→S6). La presenza del three-way handshake TCP (SYN, SYN-ACK, ACK) e dei segmenti dati con riscontro conferma che il traffico generico non attraversa il Premium Link.

No.	Time	Source	Destination	Protocol	Length	Info
153	2.018371	10.0.0.1	10.0.0.3	TCP	74	38194 → 5001 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM=1 TSval=436244647 TSecr=0 WS=512
166	2.179197	10.0.0.3	10.0.0.1	TCP	74	5001 → 38194 [ACK] Seq=8 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM=1 TSval=2468174477 TSecr=
173	2.242949	10.0.0.1	10.0.0.3	TCP	66	38194 → 5001 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=436244872 TSecr=2468174477
174	2.242951	10.0.0.1	10.0.0.3	TCP	126	38194 → 5001 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=60 TSval=436244872 TSecr=2468174477
175	2.242952	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=61 Ack=1 Win=42496 Len=2896 TSval=436244872 TSecr=2468174477
176	2.242965	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=2957 Ack=1 Win=42496 Len=2896 TSval=436244872 TSecr=2468174477
177	2.263349	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=5853 Ack=1 Win=42496 Len=2896 TSval=436244872 TSecr=2468174477
178	2.273673	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=8749 Ack=1 Win=42496 Len=2896 TSval=436244872 TSecr=2468174477
189	2.285780	10.0.0.1	10.0.0.3	TCP	1514	38194 → 5001 [ACK] Seq=11645 Ack=1 Win=42496 Len=1448 TSval=436244872 TSecr=2468174477
191	2.404043	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=1 Ack=61 Win=43520 Len=0 TSval=2468174701 TSecr=436244872
192	2.404050	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=1 Ack=2957 Win=49664 Len=0 TSval=2468174701 TSecr=436244872
193	2.404051	10.0.0.3	10.0.0.1	TCP	94	5001 → 38194 [PSH, ACK] Seq=1 Ack=2957 Win=49664 Len=28 TSval=2468174702 TSecr=436244872
194	2.410334	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=5853 Win=55296 Len=0 TSval=2468174709 TSecr=436244872
195	2.423911	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=8749 Win=60928 Len=0 TSval=2468174722 TSecr=436244872
197	2.434559	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=11645 Win=62976 Len=0 TSval=2468174733 TSecr=436244872
202	2.460647	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=13093 Ack=1 Win=42496 Len=2896 TSval=436245096 TSecr=2468174701
203	2.473661	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=15989 Ack=1 Win=42496 Len=2896 TSval=436245096 TSecr=2468174701
204	2.480453	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=18885 Ack=1 Win=42496 Len=2896 TSval=436245096 TSecr=2468174701
206	2.492535	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=13093 Win=64080 Len=0 TSval=2468174791 TSecr=436244872
207	2.497055	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=11645 Win=62976 Len=0 TSval=436245096 TSecr=2468174702
208	2.497061	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=21781 Ack=29 Win=42496 Len=2896 TSval=436245102 TSecr=2468174709
210	2.508427	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=24677 Ack=29 Win=42496 Len=2896 TSval=436245102 TSecr=2468174709
211	2.521878	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=27573 Ack=29 Win=42496 Len=2896 TSval=436245116 TSecr=2468174722
213	2.533466	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=30469 Ack=29 Win=42496 Len=2896 TSval=436245116 TSecr=2468174722
214	2.547295	10.0.0.1	10.0.0.3	TCP	1514	38194 → 5001 [PSH, ACK] Seq=33365 Ack=29 Win=42496 Len=1448 TSval=436245128 TSecr=2468174733
215	2.552135	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=34813 Ack=29 Win=42496 Len=2896 TSval=436245128 TSecr=2468174733
220	2.564598	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=37709 Ack=29 Win=42496 Len=2896 TSval=436245184 TSecr=2468174791
222	2.578797	10.0.0.1	10.0.0.3	TCP	1514	38194 → 5001 [ACK] Seq=40605 Ack=29 Win=42496 Len=1448 TSval=436245184 TSecr=2468174791
226	2.633533	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=15989 Win=62976 Len=0 TSval=2468174929 TSecr=436245096
227	2.640627	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=18885 Win=62976 Len=0 TSval=2468174937 TSecr=436245096
229	2.654061	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=21781 Win=69632 Len=0 TSval=2468174950 TSecr=436245096
234	2.665398	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=24677 Win=75776 Len=0 TSval=2468174963 TSecr=436245102
235	2.686577	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=27573 Win=81408 Len=0 TSval=2468174978 TSecr=436245102
236	2.693228	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=30469 Win=87040 Len=0 TSval=2468174990 TSecr=436245116
237	2.698174	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=42053 Ack=29 Win=42496 Len=2896 TSval=436245325 TSecr=2468174929
238	2.698302	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=33365 Win=93184 Len=0 TSval=2468174990 TSecr=436245116
239	2.704119	10.0.0.1	10.0.0.3	TCP	2962	38194 → 5001 [PSH, ACK] Seq=44949 Ack=29 Win=42496 Len=2896 TSval=436245325 TSecr=2468174929
240	2.714386	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=34813 Win=95232 Len=0 TSval=2468175009 TSecr=436245128
242	2.715543	10.0.0.3	10.0.0.1	TCP	66	5001 → 38194 [ACK] Seq=29 Ack=37709 Win=94208 Len=0 TSval=2468175014 TSecr=436245128

Frame 153: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
 Ethernet II, Src: 00:00:00:00:00:01 (00:00:00:00:00:01), Dst: 00:00:00:00:00:03 (00:00:00:00:00:03)
 Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.3
 Transmission Control Protocol, Src Port: 38194, Dst Port: 5001, Seq: 0, Len: 0

Pacchetti: 500 - visualizzati: 227 (45.4%) Profilo: Default

Figura 24: Cattura Wireshark sul percorso standard (S2→S3): traffico TCP porta 5001 da H1 (10.0.0.1) a H3 (10.0.0.3). Il three-way handshake e i segmenti dati con ACK confermano che il traffico generico viene instradato sul link standard a 2 Mbps, separato dal traffico video.

Profilo temporale durante il dynamic slicing. Il grafico I/O di Wireshark sulla cattura del dynamic slicing mostra il profilo temporale del traffico sul Premium Link. Si osserva chiaramente il momento in cui il traffico video viene attivato (intorno al secondo 12): il packet rate sale da quasi zero a circa 350 pacchetti/secondo, corrispondente a un flusso video attivo. Il profilo rimane stabile per tutta la durata del flusso, indicando che il Premium Link gestisce il traffico senza congestione.

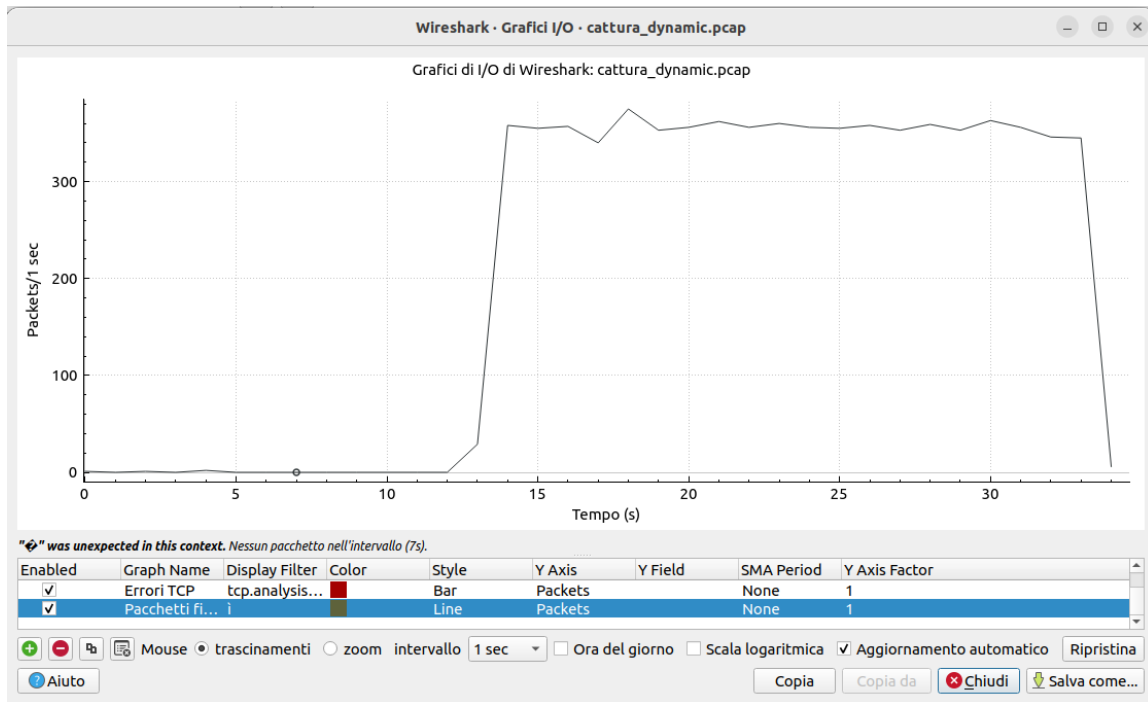


Figura 25: Grafico I/O di Wireshark durante il dynamic slicing: il packet rate sale bruscamente al secondo 12 quando il flusso video viene attivato, raggiungendo ~350 pacchetti/s. Il profilo stabile conferma l'assenza di congestione sul Premium Link durante il trasferimento video.

6.8 Grafici dei Risultati



Figura 26: Dashboard Grafana in stato idle: i sei pannelli mostrano throughput a zero sui Premium Links, nessun video attivo e contatore preemption a zero.



Figura 27: Dashboard Grafana durante un flusso video: il throughput del Premium Link sale a ~ 5 Mbps, il flag “Video Active” si attiva e il contatore di preemption viene incrementato.

6.9 Generazione Avanzata di Traffico con D-ITG

D-ITG (Distributed Internet Traffic Generator) è uno strumento per la generazione e misurazione del traffico di rete, sviluppato presso l’Università Federico II di Napoli. Rispetto a `iperf`, D-ITG offre diversi vantaggi per la caratterizzazione delle prestazioni di rete:

- Misurazione precisa del **one-way delay** (latenza unidirezionale) per ogni pacchetto, non solo RTT
- Calcolo del **jitter** (variazione del ritardo) secondo RFC 3550
- Statistiche dettagliate di **packet loss** con analisi delle burst
- Generazione di traffico con distribuzione statistica controllata (CBR, Poisson, esponenziale)

Parametri dei flussi. I test D-ITG sono eseguiti con il `dynamic_slicing_controller.py` per verificare il comportamento della preemption anche con traffico generato a livello di pacchetto.

Tabella 11: Parametri dei flussi D-ITG utilizzati nei test

Flusso	Porta	Rate	Pkt size	Durata	Banda offerta
Normale	UDP 800	500 pkt/s	1000 B	60 s	~ 4 Mbps
Video	UDP 9999	4000 pkt/s	1400 B	30 s	~ 44.8 Mbps

Il flusso video offre 44.8 Mbps su un Premium Link da 6 Mbps, generando intenzionalmente saturazione per misurare il packet loss in condizioni estreme.

Comandi utilizzati. L'architettura D-ITG prevede tre componenti: **ITGRecv** (ricevitore), **ITGSend** (generatore) e **ITGDec** (decodificatore dei log). Un singolo **ITGRecv** gestisce più flussi contemporaneamente tramite il canale di segnalazione (porta 9000).

```

1 # Avvio ricevitore su H3 (gestisce tutti i flussi)
2 H3 ITGRecv -l /tmp/ditg/recv.log &
3
4 # Flow Normale: 500 pkt/s, 1000 B, 60 s
5 H1 ITGSend -a 10.0.0.3 -rp 800 -C 500 -c 1000 -t 60000 -T UDP \
6     -l /tmp/ditg/normal_send.log &
7
8 # Flow Video: 4000 pkt/s, 1400 B, 30 s (avviato dopo 15 s)
9 H1 ITGSend -a 10.0.0.3 -rp 9999 -C 4000 -c 1400 -t 30000 -T UDP \
10     -l /tmp/ditg/video_send.log
11
12 # Analisi risultati
13 ITGDec /tmp/ditg/recv.log

```

Listing 31: Comandi D-ITG eseguiti nella CLI di Mininet

Risultati. I risultati misurati dal ricevitore (**recv.log**) mostrano le statistiche per ciascun flusso con metriche non ottenibili da **iperf**:

Tabella 12: Risultati D-ITG misurati dal ricevitore (ITGDec recv.log)

Metrica	Flow Normale (UDP 800)	Flow Video (UDP 9999)
Pacchetti ricevuti	14 506	14 058
Pacchetti persi	2 919 (16.75%)	21 514 (60.48%)
Delay medio	4.073 s	3.095 s
Delay min / max	0.112 / 4.902 s	0.017 / 3.740 s
Jitter medio	3.404 ms	2.260 ms
Bitrate medio ricevuto	1 808 Kbps	4 928 Kbps
Packet rate medio	226 pkt/s	440 pkt/s
Loss burst medio	1.06 pkt	2.16 pkt

Il flow video presenta un packet loss del 60.48%, coerente con il rapporto tra banda offerta (44.8 Mbps) e capacità del link (6 Mbps). Il flow normale, inizialmente allocato sul Premium Link (priority=110), viene spostato sul percorso standard dopo la preemption, causando un packet loss del 16.75%.

Il delay elevato (3–4 s) è dovuto all'accodamento nei buffer degli switch virtuali Open vSwitch: i pacchetti in eccesso vengono bufferizzati prima di essere scartati, introducendo latenza nell'ordine dei secondi.

Confronto D-ITG vs iperf. La tabella seguente confronta le capacità diagnostiche dei due strumenti:

Tabella 13: Confronto delle metriche disponibili tra iperf e D-ITG

Metrica	iperf	D-ITG
Throughput / bitrate	✓	✓
Packet loss (%)	✓(solo UDP)	✓
One-way delay	–	✓
Jitter (RFC 3550)	✓(aggregato)	✓(per pacchetto)
Loss burst analysis	–	✓
Delay std deviation	–	✓
Log per-pacchetto	–	✓
Analisi post-hoc (ITGDec)	–	✓

D-ITG è utile soprattutto per analizzare il comportamento della preemption: dal log per-pacchetto si può individuare l'istante in cui il flusso normale viene spostato dal Premium Link al percorso standard, osservando il salto nel delay e l'inizio delle perdite.

```
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ ITGDec "/home/vboxuser/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella/test_results/ditg/recv.log"
ITGDec version 2.8.1 (r1023)
Compile-time options: sctp dccp bursty multiport
/-----
Flow number: 1
From 10.0.0.1:42831
To 10.0.0.3:800
-----
Total time = 64.178569 s
Total packets = 14506
Minimum delay = 0.112142 s
Maximum delay = 4.902389 s
Average delay = 4.073263 s
Average jitter = 0.003404 s
Delay standard deviation = 1.008207 s
Bytes received = 14506000
Average bitrate = 1808.204854 Kbit/s
Average packet rate = 226.025607 pkt/s
Packets dropped = 2919 (16.75 %)
Average loss-burst size = 1.055696 pkt
-----
Flow number: 1
From 10.0.0.1:45287
To 10.0.0.3:9999
```

Figura 28: Output di ITGDec – Flusso Normale (UDP 800): 14506 pacchetti, 16.75% packet loss, delay medio 4.073s, bitrate 1808 Kbit/s. Il flusso normale passa inizialmente dal premium link (regola a priorità 110), poi viene spostato sul percorso standard dopo la preemption.

```

-----
Total time           =      31.950422 s
Total packets        =      14058
Minimum delay        =      0.016690 s
Maximum delay        =      3.739567 s
Average delay        =      3.094775 s
Average jitter        =      0.002260 s
Delay standard deviation =    0.759044 s
Bytes received        =    19681200
Average bitrate       =    4927.934911 Kbit/s
Average packet rate   =    439.994188 pkt/s
Packets dropped       =      21514 (60.48 %)
Average loss-burst size =    2.164169 pkt
-----

```

Figura 29: Output di ITGDec – Flusso Video (UDP 9999): 14058 pacchetti, 60.48% packet loss, delay medio 3.094 s, bitrate 4927 Kbit/s. Le perdite elevate sono dovute al rate offerto (44.8 Mbps) che eccede la capacità del premium link (6 Mbps).

```

***** TOTAL RESULTS *****
-----
Number of flows      =          2
Total time           =    101.489999 s
Total packets        =    28564
Minimum delay        =      0.016690 s
Maximum delay        =      4.902389 s
Average delay        =      3.591692 s
Average jitter        =      0.002990 s
Delay standard deviation =    1.019347 s
Bytes received        =    34187200
Average bitrate       =    2694.823162 Kbit/s
Average packet rate   =    281.446451 pkt/s
Packets dropped       =      24433 (46.10 %)
Average loss-burst size =    1.922950 pkt
Error lines           =          0
-----

```

Figura 30: Output di ITGDec – Risultati Totali: 2 flussi, 28564 pacchetti, 46.10% packet loss complessivo. Il flusso video ha occupato il premium link e il flusso normale è stato spostato sul percorso standard.

7 Dashboard e Monitoraggio

7.1 Architettura del Monitoraggio

Il sistema di monitoraggio segue un'architettura a tre livelli: raccolta dati, persistenza e visualizzazione. Ogni livello opera indipendentemente, in modo che il malfunzionamento di un componente non comprometta gli altri.

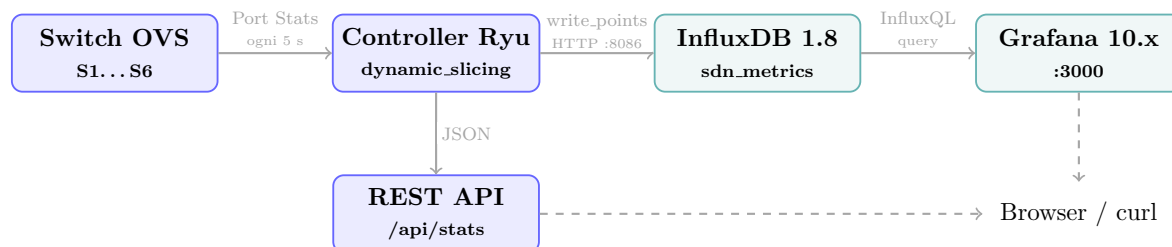


Figura 31: Architettura del monitoraggio: il controller raccoglie le statistiche degli switch ogni 5 secondi, le scrive su InfluxDB e le espone tramite REST API. Grafana interroga InfluxDB per la visualizzazione grafica.

I tre componenti:

- **Controller Ryu** — il monitor loop (`_monitor_loop`) invia richieste `OFPPortStatsRequest` a tutti gli switch ogni 5 secondi. Le risposte vengono elaborate per calcolare il throughput per-porta, e i risultati scritti su InfluxDB tramite la libreria Python `influxdb`. Il controller espone inoltre un endpoint REST `GET /api/stats` tramite WSGI, che restituisce le stesse metriche in formato JSON.
- **InfluxDB 1.8** — database a serie temporali. Ogni punto dati contiene un timestamp, tag (switch, porta, percorso) e campi numerici. Il database `sdn_metrics` viene creato al primo avvio. InfluxDB è in ascolto sulla porta 8086 (HTTP API).
- **Grafana 10.x** — piattaforma di visualizzazione con pannelli grafici aggiornati automaticamente. La dashboard è raggiungibile su `http://localhost:3000`. Grafana interroga InfluxDB con query InfluxQL e aggiorna i pannelli ogni 5 secondi.

Se InfluxDB non è disponibile all'avvio, il controller disabilita la scrittura delle metriche e prosegue con tutte le altre funzionalità intatte.

7.2 Scrittura delle Metriche nel Controller

La connessione a InfluxDB viene stabilita nel costruttore del controller. La libreria `influxdb` è importata con un blocco `try/except` per gestire il caso in cui non sia installata:

```

1 try:
2     from influxdb import InfluxDBClient as _InfluxDBClient
3 except ImportError:
4     _InfluxDBClient = None
  
```

```

5
6 # Nel costruttore __init__:
7 self.influx = None
8 if _InfluxDBClient is not None:
9     try:
10         self.influx = _InfluxDBClient('localhost', 8086,
11                                     database='sdn_metrics')
12         self.influx.ping()
13         self.logger.info("InfluxDB connesso (database: sdn_metrics)")
14     except Exception as e:
15         self.logger.warning("InfluxDB non raggiungibile: %s", e)
16         self.influx = None

```

Listing 32: Inizializzazione del client InfluxDB nel controller

Ad ogni risposta delle port stats (ogni 5 secondi), il controller costruisce i punti dati e li invia a InfluxDB con una singola chiamata `write_points`:

```

1 if self.influx is not None:
2     try:
3         points = []
4         for d, info in self.premium_links.items():
5             points.append({
6                 "measurement": "premium_link",
7                 "tags": {"switch": "s%d" % d,
8                         "path": "upper" if d == 2 else "lower"},
9                 "fields": {
10                     "usage_mbps": float(info['usage_mbps']),
11                     "capacity_mbps": float(info['capacity_mbps']),
12                     "usage_pct": float(info['usage_mbps'] /
13                                       info['capacity_mbps'] * 100),
14                     "video_active": int(info['video_active']),
15                     "dynamic_active": int(info['dynamic_active']),
16                     "preemption_count": self.preemption_count,
17                 }
18             })
19         # Velocita' per-porta di tutti gli switch
20         for d, ports in self.port_speed.items():
21             for port_no, speed in ports.items():
22                 points.append({
23                     "measurement": "port_throughput",
24                     "tags": {"switch": "s%d" % d,
25                             "port": str(port_no)},
26                     "fields": {
27                         "tx_mbps": float(speed.get('tx_mbps', 0)),
28                     }
29                 })
30             self.influx.write_points(points)
31     except Exception:
32         pass # non bloccare il controller se InfluxDB non risponde

```

Listing 33: Scrittura dei punti dati su InfluxDB (dal gestore port stats)

I punti dati si dividono in due measurement:

Tabella 14: Measurement `premium_link` — stato dei Premium Links

Campo	Tipo	Descrizione
<code>usage_mbps</code>	float	Banda utilizzata sul Premium Link (Mbps)
<code>capacity_mbps</code>	float	Capacità del Premium Link (6 Mbps)
<code>usage_pct</code>	float	Percentuale di utilizzo
<code>video_active</code>	int	1 se è presente traffico video
<code>dynamic_active</code>	int	1 se è attiva l'allocazione dinamica
<code>preemption_count</code>	int	Numero totale di preemption eseguite

Tag: `switch` (s2, s4), `path` (upper, lower).

Tabella 15: Measurement `port_throughput` — throughput per-porta

Campo	Tipo	Descrizione
<code>tx_mbps</code>	float	Throughput in trasmissione calcolato con EMA ($\alpha = 0.3$)

Tag: `switch` (s1...s6), `port` (numero porta).

7.3 Configurazione di InfluxDB

InfluxDB 1.8 viene installato come servizio di sistema. La creazione del database e la verifica della connessione si effettuano da riga di comando:

```

1 # Installazione su Ubuntu/Debian
2 sudo apt-get install influxdb
3
4 # Avvio del servizio
5 sudo systemctl start influxdb
6 sudo systemctl enable influxdb
7
8 # Verifica che il servizio risponda
9 curl -s http://localhost:8086/ping -I
10 # HTTP/1.1 204 No Content <-- InfluxDB operativo
11
12 # Creazione del database (dalla CLI di InfluxDB)
13 influx
14 > CREATE DATABASE sdn_metrics
15 > SHOW DATABASES
16 # name: databases
17 # name
18 # ----
19 # _internal
20 # sdn_metrics
21 > exit
22
23 # Installazione della libreria Python per il controller
24 pip3 install influxdb

```

Listing 34: Installazione e configurazione di InfluxDB

Il controller crea automaticamente il database `sdn_metrics` se non esiste già. La connessione avviene sulla porta 8086 di `localhost` senza autenticazione (configurazione di default di InfluxDB 1.8).

7.4 Configurazione di Grafana

Grafana viene configurato in tre passi: installazione, aggiunta del datasource InfluxDB e creazione dei pannelli della dashboard.

```

1 # Installazione su Ubuntu/Debian
2 sudo apt-get install grafana
3
4 # Avvio del servizio
5 sudo systemctl start grafana-server
6 sudo systemctl enable grafana-server
7
8 # Accesso alla dashboard
9 # Aprire http://localhost:3000 nel browser
10 # Credenziali di default: admin / admin (cambiarle al primo accesso)

```

Listing 35: Installazione e avvio di Grafana

Aggiunta del datasource InfluxDB. Nella sezione *Configuration* → *Data Sources* → *Add data source*:

- **Type:** InfluxDB
- **URL:** http://localhost:8086
- **Database:** sdn_metrics
- **HTTP Method:** GET

Query InfluxQL per i pannelli. Ogni pannello di Grafana interroga InfluxDB con una query InfluxQL. Le query utilizzate per i sei pannelli della dashboard:

```

1 # Pannello "Premium Link Usage (Mbps)"
2 SELECT mean("usage_mbps") FROM "premium_link"
3     WHERE $timeFilter GROUP BY time($__interval), "path" fill(0)
4
5 # Pannello "Premium Link Usage (%)"
6 SELECT mean("usage_pct") FROM "premium_link"
7     WHERE $timeFilter GROUP BY time($__interval), "path" fill(0)
8
9 # Pannello "Video Active"
10 SELECT last("video_active") FROM "premium_link"
11     WHERE $timeFilter GROUP BY time($__interval), "path" fill(0)
12
13 # Pannello "Preemption Count"
14 SELECT last("preemption_count") FROM "premium_link"
15     WHERE $timeFilter GROUP BY time($__interval) fill(previous)
16
17 # Pannello "Port Throughput - S2"
18 SELECT mean("tx_mbps") FROM "port_throughput"
19     WHERE "switch" = 's2' AND $timeFilter
20     GROUP BY time($__interval), "port" fill(0)
21
22 # Pannello "Port Throughput - S4" (analogo con switch = 's4')

```

Listing 36: Query InfluxQL usate nei pannelli Grafana

Le variabili `$timeFilter` e `$__interval` sono gestite automaticamente da Grafana in base all'intervallo temporale selezionato dall'utente nella dashboard.

7.5 Pannelli della Dashboard

La dashboard Grafana contiene sei pannelli, organizzati in tre righe:

- **Premium Link Usage (Mbps)** — grafico a linee del throughput dei due Premium Links (upper S2→S6 e lower S4→S6). Le due serie sono distinte dal tag path.
- **Premium Link Usage (%)** — percentuale di utilizzo della capacità (6 Mbps). La soglia di preemption all'80% è visibile come linea orizzontale sul grafico.
- **Video Active** — serie temporale booleana (0/1) che indica quando il traffico video è presente su ciascun Premium Link. I fronti di salita e discesa corrispondono all'inizio e alla fine dei flussi video.
- **Preemption Count** — contatore cumulativo delle preemption eseguite. Ogni gradino nel grafico corrisponde a un evento di preemption.
- **Port Throughput – S2** — throughput in Mbps per ogni porta dello switch S2. La porta 2 (standard) e la porta 3 (premium) permettono di osservare la distribuzione del traffico.
- **Port Throughput – S4** — analogo per S4 (percorso inferiore).

7.6 REST API

Il controller espone un endpoint REST GET `/api/stats` tramite il modulo WSGI di Ryu. L'endpoint restituisce un JSON con le stesse metriche scritte su InfluxDB, permettendo l'interrogazione diretta da strumenti esterni senza dipendere da InfluxDB:

```
1 # Query delle statistiche correnti
2 curl -s http://127.0.0.1:8080/api/stats | python3 -m json.tool
```

Listing 37: Interrogazione della REST API del controller

La risposta contiene lo stato corrente dei Premium Links, il contatore di preemption e gli ultimi eventi registrati dal controller (allocazioni, rilevamenti video, preemption).

7.7 Deployment con Docker Compose

In alternativa all'installazione manuale di InfluxDB e Grafana, il progetto include un file `docker-compose.yml` nella directory `monitoring/` che avvia l'intero stack di monitoraggio con un singolo comando. Questa soluzione garantisce riproducibilità e isolamento dall'ambiente host.

```
1 version: "3.8"
2
3 services:
4   influxdb:
5     image: influxdb:1.8
6     container_name: sdn_influxdb
7     restart: unless-stopped
8     ports:
9       - "8086:8086"
10    environment:
```

```
11     - INFLUXDB_DB=sdn_metrics
12     - INFLUXDB_HTTP_AUTH_ENABLED=false
13 volumes:
14     - influxdb_data:/var/lib/influxdb
15 networks:
16     - monitoring
17
18 grafana:
19     image: grafana/grafana:10.2.0
20     container_name: sdn_grafana
21     restart: unless-stopped
22     ports:
23     - "3000:3000"
24     environment:
25     - GF_SECURITY_ADMIN_USER=admin
26     - GF_SECURITY_ADMIN_PASSWORD=admin
27     - GF_USERS_ALLOW_SIGN_UP=false
28     volumes:
29     - grafana_data:/var/lib/grafana
30     - ./grafana/provisioning/datasources:
31       /etc/grafana/provisioning/datasources
32     - ./grafana/provisioning/dashboards:
33       /etc/grafana/provisioning/dashboards
34     - ./grafana/dashboards:
35       /etc/grafana/provisioning/dashboards/json
36     depends_on:
37     - influxdb
38     networks:
39     - monitoring
40
41 volumes:
42     influxdb_data:
43     grafana_data:
44
45 networks:
46     monitoring:
47         driver: bridge
```

Listing 38: Contenuto di monitoring/docker-compose.yml

Aspetti chiave della configurazione:

- **InfluxDB 1.8:** la variabile d'ambiente `INFLUXDB_DB=sdn_metrics` crea automaticamente il database al primo avvio, eliminando la necessità di creazione manuale tramite la CLI. La porta 8086 viene esposta all'host per consentire al controller Ryu (in esecuzione fuori dal container) di scrivere le metriche.
- **Grafana 10.2.0:** le credenziali di default (admin/admin) sono configurate tramite variabili d'ambiente. Tre volumi montano rispettivamente la configurazione dei datasource, la configurazione delle dashboard e i file JSON delle dashboard, consentendo il provisioning automatico all'avvio.
- **Network monitoring:** una rete bridge dedicata consente la comunicazione tra i container tramite il nome del servizio (`http://influxdb:8086`), isolandoli dalle altre reti Docker.

- **Volumi persistenti:** `influxdb_data` e `grafana_data` garantiscono che i dati delle metriche e le configurazioni di Grafana sopravvivano al riavvio dei container.
- **depends_on:** Grafana attende che InfluxDB sia avviato prima di partire, evitando errori di connessione al datasource durante l'inizializzazione.

L'avvio dello stack richiede un singolo comando:

```

1 # Avvio in background
2 cd monitoring
3 docker-compose up -d
4
5 # Verifica dei container
6 docker-compose ps
7 #      Name                  Command              State          Ports
8 # sdn_grafana                /run.sh              Up            0.0.0.0:3000->3000/
9 # sdn_influxdb               /entrypoint.sh influxd  Up            0.0.0.0:8086->8086/
10 #
11 # Verifica InfluxDB
12 curl -s http://localhost:8086/ping -I
13 # HTTP/1.1 204 No Content
14
15 # Grafana raggiungibile su http://localhost:3000 (admin/admin)
16
17 # Arresto dello stack
18 docker-compose down
19 # Per rimuovere anche i volumi dati: docker-compose down -v

```

Listing 39: Avvio e verifica dello stack Docker Compose

7.8 Auto-Provisioning Grafana

Grafana supporta il *provisioning* automatico: al primo avvio, legge i file YAML dalle directory montate e configura datasource e dashboard senza intervento manuale. Questo elimina la necessità di configurare Grafana dall'interfaccia web e garantisce che l'ambiente sia immediatamente operativo.

Datasource. Il file `datasource.yml` registra InfluxDB come sorgente dati predefinita:

```

1 apiVersion: 1
2
3 datasources:
4   - name: InfluxDB
5     type: influxdb
6     access: proxy
7     url: http://influxdb:8086
8     database: sdn_metrics
9     isDefault: true
10    editable: false

```

Listing 40: File `monitoring/grafana/provisioning/datasources/datasource.yml`

Il campo `url` utilizza il nome del servizio Docker (`influxdb`) come hostname, risolto automaticamente dalla rete interna Docker. L'opzione `access: proxy` indica che le query a InfluxDB passano attraverso il backend di Grafana, non direttamente dal browser dell'utente.

Dashboard provider. Il file `dashboard.yml` indica a Grafana dove trovare le dashboard JSON da caricare automaticamente:

```

1 apiVersion: 1
2
3 providers:
4   - name: SDN Network Slicing
5     orgId: 1
6     folder: ""
7     type: file
8     disableDeletion: false
9     editable: true
10    options:
11      path: /etc/grafana/provisioning/dashboards/json
12      foldersFromFilesStructure: false

```

Listing 41: File `monitoring/grafana/provisioning/dashboards/dashboard.yml`

Il percorso `/etc/grafana/provisioning/dashboards/json` corrisponde al volume montato nel `docker-compose.yml` dalla directory locale `./grafana/dashboards/`, dove risiede il file JSON della dashboard `sdn_network_slicing.json`.

Flusso di auto-provisioning. All'avvio, Grafana esegue automaticamente i seguenti passi:

1. Legge `datasource.yml` e registra il datasource InfluxDB con URL `http://influxdb:8086` e database `sdn_metrics`.
2. Legge `dashboard.yml` e individua la directory `/etc/grafana/provisioning/dashboards/`.
3. Carica `sdn_network_slicing.json` e rende disponibile la dashboard con i sei pannelli già configurati e le query InfluxQL preimpostate.

L'utente che accede a Grafana su `http://localhost:3000` trova la dashboard “SDN Network Slicing – Premium Links” già pronta con i sei pannelli operativi, senza necessità di configurazione manuale.

La struttura dei file di provisioning nella directory del progetto:

```

1 monitoring/
2 |-- docker-compose.yml
3 |-- grafana/
4   |-- dashboards/
5   |   |-- sdn_network_slicing.json      # Dashboard con 6 pannelli
6   |-- provisioning/
7   |   |-- datasources/
8   |   |   |-- datasource.yml           # Connessione InfluxDB
9   |   |-- dashboards/
10  |       |-- dashboard.yml             # Provider dashboard JSON

```

Listing 42: Struttura della directory `monitoring/`

8 Istruzioni Operative

8.1 Requisiti

Componenti software necessari:

- **Mininet** 2.3.0 o superiore – emulatore di rete SDN
- **Ryu** 4.34 o superiore – controller SDN OpenFlow
- **Python** 3.8 o superiore – interprete per controller e script
- **Open vSwitch** 2.13 o superiore – switch virtuale OpenFlow
- **iperf** – generatore di traffico per i test
- **InfluxDB** 1.8 – database a serie temporali per le metriche
- **Grafana** 10.x – piattaforma di visualizzazione dashboard
- **influxdb** (Python) – libreria client (`pip3 install influxdb`)

L'ambiente consigliato è una macchina virtuale Ubuntu 20.04 o superiore. InfluxDB e Grafana sono opzionali: il controller funziona anche senza di essi.

8.2 Avvio del Sistema

Servono due terminali: uno per il controller Ryu, uno per Mininet. Con il dynamic slicing, la dashboard Grafana è raggiungibile su `http://localhost:3000` (credenziali di default: `admin / admin`).

```
1 # Terminale 1: Avvio del controller Ryu
2 # Per topology slicing:
3 ryu-manager topology_slicing_controller.py
4
5 # Per service slicing con premium links:
6 ryu-manager service_slicing_controller.py
7
8 # Per dynamic slicing con video preemption:
9 ryu-manager dynamic_slicing_controller.py
10
11 # Terminale 2: Avvio della topologia Mininet
12 sudo python3 topology.py
```

Listing 43: Avvio del sistema

Avviata la topologia, Mininet presenta il prompt `mininet>` da cui eseguire i test.

```

*** Starting CLI:
mininet> net
H1 H1-eth1:s1-eth1
H2 H2-eth1:s1-eth2
H3 H3-eth1:s6-eth3
H4 H4-eth1:s6-eth6
s1 lo: s1-eth1:H1-eth1 s1-eth2:H2-eth1 s1-eth3:s2-eth1 s1-eth4:s4-eth1
s2 lo: s2-eth1:s1-eth3 s2-eth2:s3-eth1 s2-eth3:s6-eth2
s3 lo: s3-eth1:s2-eth2 s3-eth2:s6-eth1
s4 lo: s4-eth1:s1-eth4 s4-eth2:s5-eth1 s4-eth3:s6-eth5
s5 lo: s5-eth1:s4-eth2 s5-eth2:s6-eth4
s6 lo: s6-eth1:s3-eth2 s6-eth2:s2-eth3 s6-eth3:H3-eth1 s6-eth4:s5-eth2 s6-eth5:s4-eth3 s6-eth6:H4-eth1
c0
mininet>

```

Figura 32: Output del comando `net` in Mininet: la topologia mostra i 6 switch (S1–S6) e i 4 host (H1–H4) con le interfacce di rete assegnate e le connessioni tra i nodi.

8.3 Avvio del Monitoring

Per attivare il monitoraggio con Grafana, occorre avviare InfluxDB e Grafana prima del controller. La procedura completa:

```

1 # 1. Verificare che InfluxDB sia attivo
2 sudo systemctl start influxdb
3 curl -s http://localhost:8086/ping -I
4 # HTTP/1.1 204 No Content <-- InfluxDB operativo
5
6 # 2. Verificare che Grafana sia attivo
7 sudo systemctl start grafana-server
8 # Dashboard raggiungibile su http://localhost:3000
9
10 # 3. Avviare il controller (in un terminale separato)
11 # Il log conferma la connessione: "InfluxDB connesso (database:
12 # sdn_metrics)"
12 ryu-manager dynamic_slicing_controller.py
13
14 # 4. Avviare la topologia (in un altro terminale)
15 sudo python3 topology.py
16
17 # 5. Verificare le metriche via REST API
18 curl -s http://127.0.0.1:8080/api/stats | python3 -m json.tool

```

Listing 44: Avvio dell'infrastruttura di monitoraggio

Se il controller viene avviato senza InfluxDB, il log riporterà `InfluxDB non raggiungibile` e il monitoraggio sarà disabilitato. Tutte le altre funzionalità (slicing, preemption, REST API) restano attive.

```
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ ryu-manager dynamic_slicing_controller.py
1 Rlock(s) were not greened, to fix this error make sure you run eventlet.monkey_patch() before importing any other modules.
loading app dynamic_slicing_controller.py
loading app ryu.controller.ofp_handler
creating context wsgi
instantiating app dynamic_slicing_controller.py of DynamicSlicingController
InfluxDB connesso (database: sdn_metrics)
=====
Dynamic Slicing Controller avviato
Monitoraggio link premium ogni 5 sec
Controllo allocazione dinamica ogni 2 sec
=====
instantiating app ryu.controller.ofp_handler of OFPHandler
(43750) wsgi starting up on http://0.0.0.0:8080
```

Figura 33: Avvio del controller Ryu con il dynamic slicing: il terminale mostra la connessione degli switch e la conferma di connessione a InfluxDB.

```
Configurazione della rete:
- 6 switch OpenFlow 1.3 (S1-S6)
- 4 host: H1, H2, H3, H4
- Upper Slice: H1 <-> H3
- Lower Slice: H2 <-> H4
- Premium Links: 6 Mbps, 3ms (S2-S6, S4-S6)
- Standard Links: 2 Mbps, 50ms (S2-S3-S6, S4-S5-S6)

*** Creazione topologia Network Slicing con Premium Links
*** Host: H1 (10.0.0.1), H2 (10.0.0.2), H3 (10.0.0.3), H4 (10.0.0.4)
*** Switch: S1-S6 (OpenFlow 1.3)
*** Upper slice: H1-S1-S2-S3-S6-H3 (bottleneck 2 Mbps @ S2-S3, S3-S6)
*** Lower slice: H2-S1-S4-S5-S6-H4 (bottleneck 2 Mbps @ S4-S5, S5-S6)
*** Premium Links: S2-S6 e S4-S6 (6 Mbps, 3ms)
*** Creating network
*** Adding controller
*** Adding hosts:
H1 H2 H3 H4
*** Adding switches:
s1 s2 s3 s4 s5 s6
*** Adding links:
(15.00Mbit 1ms delay) (15.00Mbit 1ms delay) (H1, s1) (15.00Mbit 1ms delay) (15.00Mbit 1ms delay) (H2, s1) (15.00Mbit 1ms delay)
(15.00Mbit 5ms delay) (15.00Mbit 5ms delay) (s1, s2) (15.00Mbit 5ms delay) (15.00Mbit 5ms delay) (s1, s4) (2.00Mbit 50ms delay)
(2.00Mbit 50ms delay) (s2, s3) (6.00Mbit 3ms delay) (6.00Mbit 3ms delay) (s2, s6) (2.00Mbit 50ms delay) (2.00Mbit 50ms delay)
(2.00Mbit 50ms delay) (s3, s6) (2.00Mbit 50ms delay) (2.00Mbit 50ms delay) (s4, s5) (6.00Mbit 3ms delay) (6.00Mbit 3ms delay)
(s4, s6) (2.00Mbit 50ms delay) (2.00Mbit 50ms delay) (s5, s6) (8.00Mbit 5ms delay) (8.00Mbit 5ms delay) (s6, H3) (8.00Mbit 5ms delay)
(8.00Mbit 5ms delay) (s6, H4)
```

Figura 34: Avvio della topologia Mininet (parte 1): creazione di host, switch e link con i parametri di banda e latenza configurati.

```

*** Configuring hosts
H1 H2 H3 H4
*** Starting controller
c0
*** Starting 6 switches
s1 s2 s3 s4 s5 s6 ... (15.00Mbit 1ms delay) (15.00Mbit 1ms delay) (15.00Mbit 5ms delay) (15.00Mbit 5ms delay)
(15.00Mbit 5ms delay) (2.00Mbit 50ms delay) (6.00Mbit 3ms delay) (2.00Mbit 50ms delay) (2.00Mbit 50ms delay)
(15.00Mbit 5ms delay) (2.00Mbit 50ms delay) (6.00Mbit 3ms delay) (2.00Mbit 50ms delay) (2.00Mbit 50ms dela
y) (6.00Mbit 3ms delay) (2.00Mbit 50ms delay) (6.00Mbit 3ms delay) (2.00Mbit 50ms delay) (8.00Mbit 5ms delay)
(8.00Mbit 5ms delay)

*** Topologia avviata
*** Controller remoto: 127.0.0.1:6653

*** Comandi utili:
pingall                                - Test connettivita
H3 iperf -s -u -p 9999 &                - Server video
H1 iperf -c 10.0.0.3 -u -p 9999 -b 5M -t 10 - Test video
H3 iperf -s -u -p 5001 &                - Server dati
H1 iperf -c 10.0.0.3 -u -p 5001 -b 5M -t 10 - Test standard

*** Starting CLI:
mininet>

```

Figura 35: Avvio della topologia Mininet (parte 2): gli switch si connettono al controller e il prompt mininet> conferma che la rete è operativa.

```

vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella$ cd monitoring
vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella/monitoring$ sudo docker-compose ps

```

Name	Command	State	Ports
sdn_grafana	/run.sh	Up	0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
sdn_influxdb	/entrypoint.sh influxd	Up	0.0.0.0:8086->8086/tcp, :::8086->8086/tcp

Figura 36: Avvio dello stack Docker Compose: i container `sdn_influxdb` e `sdn_grafana` sono attivi con le porte 8086 (InfluxDB) e 3000 (Grafana) correttamente esposte all'host.

```

vboxuser@NCIS:~/Scrivania/Progetto_NCIS/Progetto_Palumbo_Sagnella/monitoring$ sudo docker exec sdn_influxdb
influx -database sdn_metrics -execute "SHOW MEASUREMENTS"
name: measurements
name
----
port_throughput
premium_link

```

Figura 37: Query InfluxDB che mostra i measurement presenti nel database `sdn_metrics`: `port_throughput` e `premium_link`, confermando che il controller scrive correttamente le metriche raccolte.

8.4 Test del Preemption

Per testare il meccanismo di video preemption, si segue questa procedura:

```

1 # 1. Avviare il controller dynamic slicing
2 ryu-manager dynamic_slicing_controller.py
3
4 # 2. Avviare la topologia
5 sudo python3 topology.py
6
7 # 3. Sulla CLI di Mininet, avviare traffico dinamico (UDP porta 800)
8 #    Questo traffico verra' allocato sui premium links
9 mininet> H3 iperf -s -u -p 800 &
10 mininet> H1 iperf -c 10.0.0.3 -u -p 800 -b 1M -t 60 &
11
12 # 4. Attendere 10 secondi per la stabilizzazione

```

```
13 # Verificare nei log: "Allocazione dinamica attivata"
14
15 # 5. Avviare il flusso video (UDP porta 9999)
16 # Questo triggera la preemption del traffico dinamico
17 mininet> H3 iperf -s -u -p 9999 &
18 mininet> H1 iperf -c 10.0.0.3 -u -p 9999 -b 4M -t 30
19
20 # 6. Osservare nei log: "Preemption eseguita" e "Flusso video rilevato"
21 # 7. Dopo 30 secondi il video termina, osservare:
22 # "Flusso video terminato" e "Allocazione dinamica attivata"
```

Listing 45: Test completo del dynamic slicing

I log del controller riportano ogni passaggio di stato, permettendo di seguire l'intero ciclo di preemption.

9 Considerazioni Finali

9.1 Risultati Ottenuti

La rete SDN emulata opera correttamente nelle tre modalità previste — topology slicing, service slicing con Premium Links e dynamic slicing con video preemption — passando dall'una all'altra avviando il controller corrispondente.

I test confermano che i Premium Links offrono circa $3\times$ la banda e latenza ridotta rispetto ai percorsi standard. Il meccanismo di preemption sposta il traffico generico dai Premium Links entro 3–5 secondi dal rilevamento del video, e il ripristino avviene entro 5–8 secondi dalla fine del flusso.

9.2 Possibili Estensioni

- **Classi di traffico multiple:** supportare più livelli di QoS (voce, video, best-effort) con soglie e priorità differenziate.
- **Scalabilità:** validare il sistema su topologie più estese, misurando l'overhead del controller al crescere del numero di switch e flussi.
- **Fault tolerance:** rilevamento dei guasti e rerouting automatico in caso di indisponibilità di un Premium Link.
- **QoS nativo OVS:** combinare la preemption con le code e il rate limiting di Open vSwitch per un controllo più fine sulla banda.

Appendice

Struttura del Progetto

```
1 SDN_Network_Slicing/  
2 |-- topology.py                # Definizione topologia Mininet  
3 |-- topology_slicing_controller.py # Controller per topology slicing  
4 |-- service_slicing_controller.py # Controller per service slicing  
5 |-- dynamic_slicing_controller.py # Controller per dynamic slicing  
   + REST API  
6 |-- monitoring/  
7 |   |-- docker-compose.yml      # Stack InfluxDB + Grafana  
8 |   |-- grafana/  
9 |     |-- dashboards/  
10 |       |-- sdn_network_slicing.json # Dashboard con 6 pannelli  
11 |       |-- provisioning/  
12 |         |-- datasources/datasource.yml # Connessione InfluxDB  
13 |         |-- dashboards/dashboard.yml   # Auto-discovery dashboard  
14 |-- documentazione/  
15 |   |-- main.tex                 # Questo documento  
16 |-- README.md                   # Istruzioni rapide
```

Listing 46: Organizzazione dei file del progetto